

Securitisation Risk Analytics Platform

Enterprise Banking Risk Analytics using SQL, Python and Microsoft Power BI

Garvit Mehta

July 2026

Contents

SECURITISATION RISK ANALYTICS PLATFORM	1
Certificate of Completion	2
Declaration	3
Acknowledgement	4
Abstract	5
Executive Summary	6
Table of Contents	8
List of Figures	9
List of Tables	10
10. Banking Industry Background	11
11. Problem Statement	13
12. Objectives	14
13. System Architecture	15
14. Dataset Description	18
15. Database Design	22
16. SQL Implementation	24
17. Python Analytics Layer	27
18. Power BI Implementation	31
19. Business Insights	42
20. Challenges	44
21. Future Enhancements	45
22. Conclusion	46
23. References	47
24. Appendix	48

SECURITISATION RISK ANALYTICS PLATFORM

Enterprise Banking Risk Analytics using SQL, Python and Microsoft Power BI

A Technical Implementation Report

Author: Garvit Mehta **Technology Stack:** MySQL · Python (Pandas, NumPy, OpenPyXL) · Microsoft Power BI · DAX · Power Query **Version:** 1.0 **Date:** July 2026

Enterprise Analytics Portfolio Documentation

Certificate of Completion

This is to certify that the project titled “**Securitisation Risk Analytics Platform**” is an original body of work, independently designed, developed, and documented by **Garvit Mehta** as a self-directed enterprise analytics initiative.

The project demonstrates applied competency across relational database design, Python-based quantitative risk modelling, and business intelligence dashboard engineering, structured to reflect the analytical and reporting standards used by asset-backed securitisation desks, credit risk teams, and investor reporting functions within retail banking and non-banking financial institutions.

The work covers the complete lifecycle of a securitised auto-loan portfolio analytics function — from raw data ingestion and validation, through IFRS 9 Expected Credit Loss computation and macroeconomic stress testing, to executive and investor-facing Power BI reporting.

Garvit Mehta B.Tech, Computer Science Engineering (Data Analytics & Data Engineering)
KIIT University

Declaration

I, **Garvit Mehta**, declare that the project “**Securitisation Risk Analytics Platform**” is a product of my own independent effort. All datasets used are synthetically generated for the explicit purpose of simulating a realistic securitised auto-loan portfolio; no proprietary, confidential, or third-party institutional data has been used at any stage of this project.

All SQL scripts, Python modules, DAX measures, and Power BI report logic contained within this project and referenced in this document were authored by me. Where industry frameworks — such as IFRS 9, Basel capital adequacy concepts, or standard securitisation cash-flow waterfall mechanics — are referenced, they are used strictly as publicly available accounting and regulatory standards, cited appropriately in the References section.

This project is submitted as a demonstration of technical and analytical capability for academic, portfolio, and professional recruitment purposes.

Acknowledgement

I would like to acknowledge the publicly available technical documentation and standards bodies that made the rigorous construction of this platform possible — namely the IFRS Foundation for the IFRS 9 financial instruments standard, the Basel Committee on Banking Supervision for capital and credit risk frameworks, and the open documentation maintained by MySQL, Python, Pandas, and Microsoft Power BI.

I also acknowledge the broader community of credit risk practitioners, structured finance analysts, and data engineers whose published methodologies on static pool vintage analysis, dynamic loss curves, and cash-flow waterfall modelling informed the design decisions taken throughout this platform.

This project was conceived, built, tested, and documented independently as part of a continuous effort to develop production-grade competency in banking risk analytics, data engineering, and business intelligence.

Abstract

Securitisation is one of the most structurally complex instruments in modern banking — it converts a static, illiquid pool of consumer loans into a tradable, tranch security whose performance must be tracked, modelled, and reported to investors on a monthly cycle for the life of the transaction, frequently five to seven years for auto-loan-backed paper. Institutions that originate, service, or invest in these structures maintain dedicated analytics functions whose sole purpose is to monitor pool performance, compute regulatory provisions, and produce investor and regulatory reporting on a fixed, auditable cadence. The **Securitisation Risk Analytics Platform** is an independently engineered simulation of exactly this function, built to reflect the technical architecture, data models, and analytical outputs that a bank or NBFC risk analytics team would produce in practice.

The platform is architected across three integrated layers. At the foundation sits a normalized MySQL relational database comprising four fact tables — loan-level portfolio data, delinquency (DPD) snapshot history, dynamic monthly loss data, and static-pool vintage curves — supported by six analytical views and three parameterised stored procedures that encapsulate reusable business logic directly at the data layer. Above this sits a Python analytics layer of three purpose-built modules: a data validation engine that enforces referential, statistical, and business-rule integrity across all four datasets before any computation is permitted to run; an IFRS 9 Expected Credit Loss engine that computes Probability of Default (PD), Loss Given Default (LGD), and Exposure at Default (EAD) driven provisioning at loan level and rolls it up into stage-wise, regional, and risk-tier summaries; and a macroeconomic stress testing engine that re-prices the entire portfolio's ECL under five calibrated scenarios ranging from a mild recession to a 2008-style financial crisis and a pandemic-shock scenario. The presentation layer consists of nine interconnected Power BI dashboards — Executive Summary, Portfolio Overview, IFRS 9, DPD & Roll Rate, Vintage, Dynamic Loss, Waterfall, Stress Testing, and Investor Reporting — each built with purpose-specific DAX measures, cross-filtering slicers, and KPI cards designed for a specific stakeholder audience, from the CRO's desk to the trustee report sent to noteholders.

Beyond its technical construction, this project is deliberately designed to demonstrate the full analytical reasoning chain that separates a reporting exercise from a risk analytics function: why a loan migrates between IFRS 9 stages, why a roll-rate matrix is a leading indicator that a static delinquency snapshot cannot provide, why the cumulative net loss curve of a 2021-Q1 vintage must be benchmarked against later vintages to detect underwriting drift, and why a stress-tested ECL uplift of 2.0x under a financial-crisis scenario has direct capital-adequacy implications for the originating institution. The document that follows walks through every layer of this build in full technical and business depth — the database schema and its normalization logic, every SQL view and stored procedure, the internals and formulae of each Python module, the KPI and DAX design of every dashboard, and twenty specific business insights the platform is capable of surfacing — structured to serve as both a standalone technical reference and a demonstration of end-to-end banking analytics capability suitable for data analyst, data engineer, and risk analytics interview evaluation.

Executive Summary

Business Problem

Retail auto-loan securitisation desks and their investors face a structural information problem: a securitised pool is a static basket of loans whose composition never changes, but the *performance* of that basket evolves every single month as borrowers pay, prepay, delinquent, cure, default, or get repossessed against. Regulatory frameworks (IFRS 9 in most jurisdictions outside the US, CECL domestically) require institutions to hold forward-looking loss provisions against this evolving risk, not simply react to losses after they occur. At the same time, the investors who bought the Class A and Class B notes issued against the pool are contractually entitled to a monthly trustee report showing collections, losses, credit enhancement levels, and pool factor — all of which must reconcile precisely across the servicer’s loan-level system, the bank’s risk engine, and the investor reporting package. In most institutions, these three views are produced by three different teams using three different tools, reconciled manually, and delivered under time pressure at each month-end close. This is slow, error-prone, and creates exactly the kind of reporting lag that regulators and rating agencies penalise.

Industry Context

Auto-loan securitisation is a multi-hundred-billion-dollar annual issuance market globally, and in India specifically it has become one of the fastest-growing structured finance products as NBFCs and banks use it to recycle balance sheet capacity. Every deal, regardless of size, requires the same core analytical machinery: static pool vintage curves to benchmark underwriting quality across origination cohorts, dynamic loss and prepayment metrics (CPR, CDR, excess spread) to track pool health in real time, a delinquency roll-rate matrix to model transition probabilities between DPD buckets, an IFRS 9 or CECL provisioning engine to compute expected credit loss under the current macro environment, macroeconomic stress testing to satisfy both internal risk appetite and regulatory stress-test obligations, and a cash-flow waterfall model to allocate collections across the credit enhancement structure — senior fees, Class A and Class B tranches, and residual distribution to the equity/first-loss piece. These are not optional analytical nice-to-haves; they are the minimum operating toolkit for any credit risk, structured finance, or investor reporting analyst working in this space.

Solution

This platform was built to replicate that toolkit end-to-end, with every component engineered rather than mocked. A normalized MySQL schema stores the four core datasets with proper primary/foreign key relationships and indexing strategy. Three Python modules implement, respectively, a pre-computation data quality gate, the IFRS 9 ECL formula ($PD \times LGD \times EAD$) applied at loan level with full stage and risk-tier rollups, and a five-scenario stress test that re-prices ECL under multiplied PD and LGD assumptions. Nine Power BI dashboards consume this data to deliver purpose-built views for portfolio managers, credit risk officers, the CRO’s executive team, and the investor reporting function — each with dedicated KPIs, DAX measures, and interactive slicers for stage, delinquency status, and servicer.

Technology

The technology stack was deliberately chosen to mirror what a mid-sized bank or NBFC’s analytics team would realistically use: MySQL as the relational backbone (universally available, ANSI-SQL compliant, trivially portable to any cloud RDBMS), Python with Pandas/NumPy/OpenPyXL for the quantitative modelling layer (the de facto standard for risk quant work outside of proprietary vendor platforms), and Microsoft Power BI for the presentation layer (the dominant BI tool inside banking and financial services organisations, ahead of Tableau and Looker in enterprise adoption within the sector). No cloud infrastructure,

proprietary risk vendor platform (e.g. Moody's RiskCalc, SAS), or paid licensing was used at any stage — the entire platform runs on freely available tooling, which was a deliberate constraint to keep the build fully reproducible.

Business Value

For an institution, a platform of this shape collapses what is typically a multi-team, multi-day monthly close process into a single reproducible pipeline: validated data flows into a provisioning engine, which flows into stress testing, which flows into a governed set of dashboards that every stakeholder — from the CRO to the trustee — draws from the same reconciled source. This eliminates the three-version-of-the-truth problem, shortens the reporting cycle, and creates full auditability: every ECL figure on every dashboard can be traced back to a specific SQL view, a specific Python calculation, and a specific validated input row.

Expected Outcomes

The completed platform demonstrates, in a single reproducible codebase, the ability to design a normalized risk-analytics database schema; implement IFRS 9 provisioning and macroeconomic stress testing exactly per the underlying accounting and regulatory logic; and build a suite of interconnected BI dashboards that would be immediately recognisable to a credit risk, structured finance, or investor reporting professional. It is built as a direct, evidence-based answer to the question every banking analytics interview eventually asks: *“Walk me through how you’d build this.”*

Table of Contents

1. Cover Page
2. Certificate of Completion
3. Declaration
4. Acknowledgement
5. Abstract
6. Executive Summary
7. Table of Contents
8. List of Figures
9. List of Tables
10. Banking Industry Background
11. Problem Statement
12. Objectives
13. System Architecture
14. Dataset Description
15. Database Design
16. SQL Implementation
17. Python Analytics Layer
18. Power BI Implementation
19. Business Insights
20. Challenges
21. Future Enhancements
22. Conclusion
23. References
24. Appendix

List of Figures

Figure	Title
Fig. 1	High-Level System Architecture
Fig. 2	Data Pipeline Flow (Raw Data → Dashboards)
Fig. 3	Entity-Relationship Diagram — Core Schema
Fig. 4	Python Analytics Layer Sequence Diagram
Fig. 5	IFRS 9 Staging Decision Flow
Fig. 6	Stress Testing Scenario Comparison Flow
Fig. 7	Cash Flow Waterfall Structure
Fig. 8	DPD Roll Rate Transition Model
Fig. 9	Static Pool Vintage Curve Logic
Fig. 10	Executive Summary Dashboard (Screenshot)
Fig. 11	Portfolio Overview Dashboard (Screenshot)
Fig. 12	IFRS-9 Dashboard (Screenshot)
Fig. 13	DPD & Roll Rate Dashboard (Screenshot)
Fig. 14	Vintage Dashboard (Screenshot)
Fig. 15	Dynamic Loss Dashboard (Screenshot)
Fig. 16	Waterfall Dashboard (Screenshot)
Fig. 17	Stress Testing Dashboard (Screenshot)
Fig. 18	Investor Reporting Dashboard (Screenshot)

List of Tables

Table	Title
Table 1	Dataset Overview Summary
Table 2	fact_auto_loans Data Dictionary
Table 3	fact_dpd_history Data Dictionary
Table 4	fact_dynamic_loss Data Dictionary
Table 5	fact_vintage Data Dictionary
Table 6	SQL View Inventory
Table 7	Stored Procedure Inventory
Table 8	IFRS 9 Risk Category Bands
Table 9	Stress Testing Scenario Calibration
Table 10	Dashboard-to-Stakeholder Mapping
Table 11	Business Insights Summary
Table 12	Technology Stack Summary

10. Banking Industry Background

10.1 Retail Lending and the Auto Loan Product

Retail auto lending is a secured consumer credit product in which a bank or NBFC advances funds to a borrower for vehicle purchase, taking a lien on the vehicle as collateral. Structurally it sits between unsecured personal lending (higher risk, higher yield, no collateral) and mortgage lending (larger ticket size, longer tenor, real-estate collateral). Auto loans typically carry tenors of three to seven years, loan-to-value (LTV) ratios in the 70-90% range at origination, and pricing that reflects both the borrower's credit profile (captured via bureau scores such as CIBIL in the Indian market) and the depreciating nature of the underlying collateral. Because the collateral value declines faster than the loan amortises in the early months of the loan (a phenomenon sometimes referred to as being "upside down" or negative equity), auto loan portfolios exhibit a distinctive risk profile: default frequency is relatively higher than mortgages, but loss severity is bounded by the resale value of a repossessed vehicle, giving portfolio-level loss rates that are more volatile but individually smaller in magnitude.

10.2 Securitisation

Securitisation is the process by which a pool of individually illiquid loans — in this case, thousands of individual auto loan contracts — is aggregated and sold to a bankruptcy-remote legal entity, which then issues tradable securities backed by the cash flows of that pool. The originating bank or NBFC benefits by removing the loans (and their associated capital charge) from its own balance sheet and receiving upfront liquidity, which it can redeploy into new originations. Investors benefit by gaining exposure to a diversified pool of consumer credit risk with a defined, tranchised risk-return profile, rather than needing to underwrite thousands of individual loans themselves.

10.3 Asset-Backed Securities (ABS) and the Special Purpose Vehicle (SPV)

The securities issued against the pool are called Asset-Backed Securities. They are issued not by the originating bank directly but by a Special Purpose Vehicle (SPV) — a legally separate entity created solely to hold the loan pool and issue notes against it. This legal separation is what makes the structure "bankruptcy remote": even if the originating bank were to fail, the SPV and its underlying loan pool are legally ring-fenced from the originator's creditors, protecting the ABS investors. The SPV appoints a trustee to oversee the structure on behalf of investors and a servicer (frequently the originating bank itself, continuing in a fee-for-service capacity) to continue collecting payments from the underlying borrowers.

10.4 Credit Enhancement and Tranching

Not all investors in an ABS deal bear the same risk. The notes issued by the SPV are typically split into tranches — commonly a senior Class A tranche and a subordinated Class B (or mezzanine) tranche, with a residual "equity" or first-loss piece typically retained by the originator. Cash collected from the pool each month is distributed according to a strict payment priority (the "waterfall," discussed below): senior fees and Class A interest and principal are paid first, Class B second, and any residual only after both classes are satisfied. This structural subordination is a form of credit enhancement — losses in the pool must exceed the size of the equity and Class B tranches before Class A investors are impaired at all, which is precisely why Class A notes typically carry investment-grade ratings even when the underlying loans, viewed individually, would not.

10.5 The Cash Flow Waterfall

The waterfall is the mechanical set of rules governing how monthly collections are distributed. A representative order of priority, and the one modelled in this platform's Waterfall Dashboard, runs: (1) trustee and servicing fees, (2) Class A interest, (3) Class A principal (until fully retired, or pro-rata depending on structure), (4) Class B interest, (5) Class B principal, (6) any residual distributed to the equity holder. Recoveries on defaulted loans are typically pooled into the collections account alongside scheduled principal, interest, and prepayments before the waterfall is applied. Understanding this waterfall is essential to interpreting investor reporting: a note's ability to pay is not a function of overall portfolio performance in isolation, but of where that note sits in the payment priority relative to realised losses.

10.6 Loan Performance States: Delinquency, Default, and Recovery

Loan performance is tracked through a delinquency (DPD — Days Past Due) lifecycle: Current, 1-29 DPD, 30-59 DPD, 60-89 DPD, 90-119 DPD, 120+ DPD, and ultimately Default or Repossession. Movement through these buckets is not always forward-only — a delinquent borrower can "cure" back to Current or a lower bucket, which is precisely what a roll-rate matrix is designed to quantify. When a loan is deemed unrecoverable through normal collection and the vehicle is repossessed, the institution realises a loss equal to the outstanding balance less the net proceeds from vehicle resale (the "recovery"). The interplay of default frequency and recovery severity is what ultimately determines the pool's net loss rate — the single most-watched metric by both risk management and ABS investors.

10.7 Investors and Investor Reporting

ABS investors — typically institutional buyers such as insurance companies, pension funds, mutual funds, and banks' own treasury desks — rely on a monthly trustee/servicer report to monitor pool health for the life of the transaction. This report must reconcile pool factor (remaining balance as a fraction of original balance), collections, losses, delinquency, and credit enhancement levels precisely, because these figures directly determine whether contractual triggers (such as early amortisation triggers tied to excess spread or cumulative loss thresholds) have been breached. The Investor Reporting Dashboard in this platform is built to replicate exactly this function.

11. Problem Statement

11.1 Current Industry Challenges

Securitisation risk analytics functions across the industry share a common set of operational pain points, each of which this platform was explicitly designed to address:

Manual, Spreadsheet-Driven Reporting. In many mid-sized institutions, the monthly ECL and investor reporting cycle still runs substantially through Excel workbooks maintained by individual analysts. This creates key-person risk, makes version control nearly impossible, and introduces manual-entry error at exactly the point in the process — regulatory provisioning — where accuracy matters most.

Regulatory Compliance Complexity. IFRS 9 requires forward-looking, stage-based provisioning rather than the incurred-loss model it replaced. Institutions must be able to demonstrate not just the ECL figure itself but the full audit trail of how PD, LGD, and EAD were derived and how loans were staged — a burden that manual processes struggle to meet under audit and regulatory examination.

Fragmented Portfolio Monitoring. Delinquency tracking, vintage analysis, and dynamic loss monitoring are frequently owned by different teams using different tools, with no single reconciled source of truth. This makes it difficult to answer a question as basic as “is this month’s loss increase driven by a specific vintage cohort or a broad macro deterioration?” without a multi-day cross-team data-gathering exercise.

Delayed and Inflexible Investor Reporting. Trustee and investor reports are contractually due on a fixed monthly schedule. When the underlying data pipeline is manual, institutions run real timeliness risk, and any late or restated report can trigger reputational and, in severe cases, contractual consequences.

Reactive, Not Scenario-Driven, Decision Making. Without an integrated stress testing capability tied directly to the live portfolio, risk committees are often making capital and pricing decisions based on point-in-time ECL figures rather than a forward-looking view of how that ECL would behave under adverse macroeconomic conditions — precisely the blind spot IFRS 9’s forward-looking design was meant to close.

11.2 The Core Problem This Platform Solves

There is no single, reconciled, end-to-end pipeline — from raw loan-level data to validated, provisioned, stress-tested, and dashboarded output — that a mid-sized institution’s risk analytics function can stand up quickly, audit fully, and extend without re-architecting. This platform demonstrates that such a pipeline can be built entirely on freely available, enterprise-standard tooling (MySQL, Python, Power BI), with every computational step traceable back to a specific script, table, or view.

12. Objectives

12.1 Business Objectives

- Provide a single reconciled source of truth for portfolio balance, expected credit loss, delinquency, and investor-facing metrics.
- Reduce the operational burden and error surface of the monthly risk and investor reporting cycle.
- Enable risk committees to make capital and pricing decisions informed by forward-looking, scenario-based ECL rather than static point-in-time figures.
- Demonstrate full audit traceability from raw data to reported KPI, satisfying the evidentiary standard expected under regulatory examination.

12.2 Functional Objectives

- Validate all four core datasets (Auto Loans, DPD History, Dynamic Loss, Vintage) against referential, statistical, and business-rule integrity checks before any downstream computation runs.
- Compute loan-level IFRS 9 Expected Credit Loss using $PD \times LGD \times EAD$, with full stage-wise, regional, and risk-tier aggregation.
- Re-price the entire portfolio's ECL under five calibrated macroeconomic stress scenarios.
- Deliver nine purpose-built Power BI dashboards covering executive, portfolio, provisioning, delinquency, vintage, loss, waterfall, stress, and investor reporting use cases.

12.3 Non-Functional Objectives

- Build entirely on free, enterprise-representative tooling (MySQL, Python, Power BI Desktop) with zero paid licensing dependency.
- Ensure the SQL schema is normalized, indexed, and portable to any ANSI-SQL-compliant cloud RDBMS with minimal modification.
- Ensure every Python module fails loudly and specifically (missing columns, invalid data) rather than silently producing incorrect output.
- Design every Power BI dashboard to be filter-consistent — a slicer selection on IFRS9_Stage or DelinquencyStatus must cross-filter identically across every visual on that page.

12.4 Technical Objectives

- Design a star-schema-oriented fact table structure with correct primary/foreign key constraints and index strategy for query performance.
- Encapsulate reusable aggregation logic in SQL views and parameterised stored procedures rather than duplicating logic across the BI layer.
- Implement stress testing as a parameterisable, scenario-driven engine rather than hard-coded one-off calculations, so new scenarios can be added without touching core logic.

13. System Architecture

13.1 Architectural Philosophy

The platform is deliberately layered so that each layer has one job and one job only, mirroring how a real institution separates data engineering, quantitative risk modelling, and business intelligence into distinct functions with distinct owners. Raw CSV extracts land first; nothing touches the risk engine until the validation layer signs off; nothing reaches a dashboard that was not computed by the Python or SQL layer. This separation is what makes the platform auditable end-to-end.

13.2 High-Level Architecture

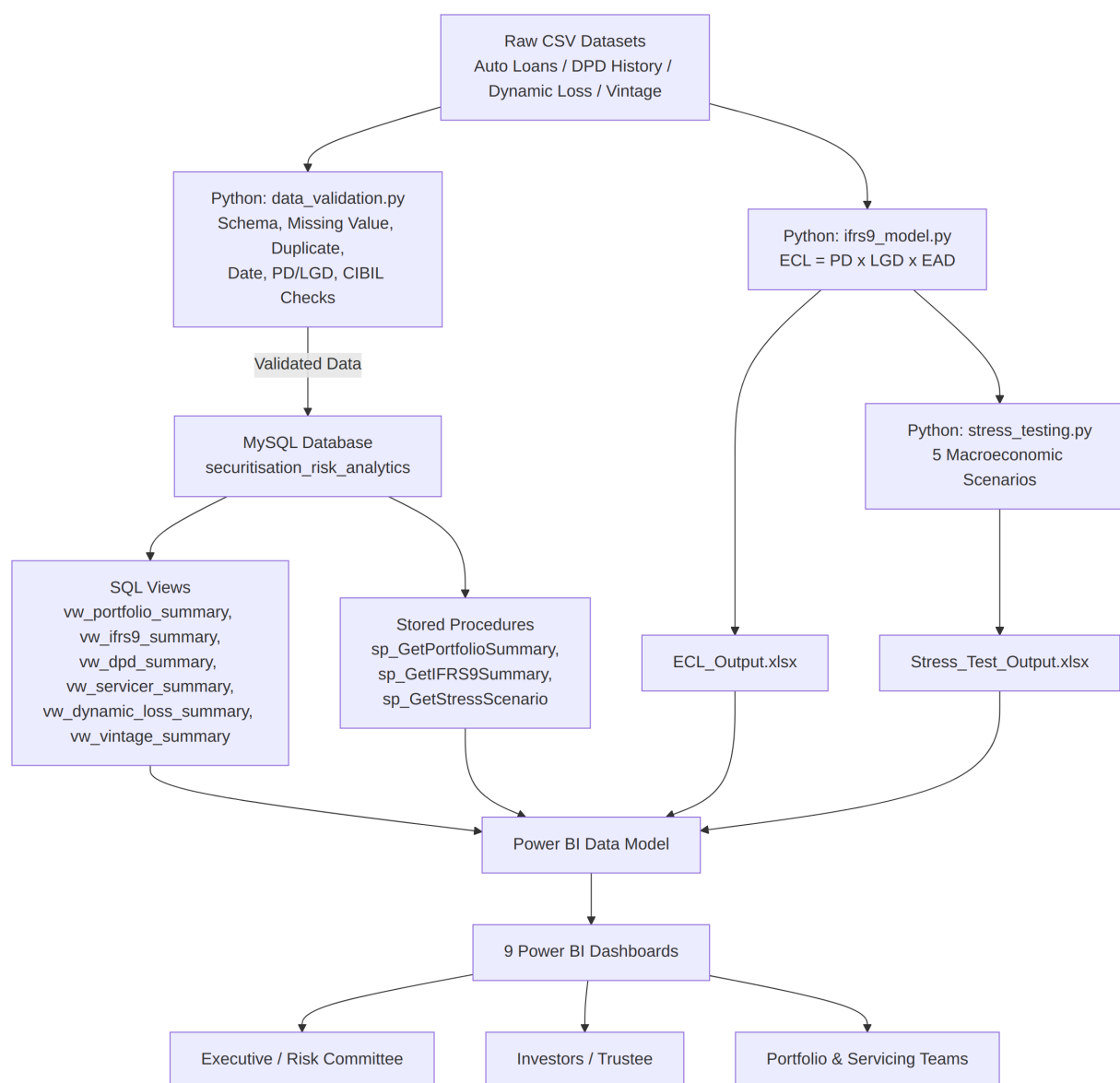


Figure 1: Diagram 1

Fig. 1 — High-Level System Architecture. Raw data is validated before it is permitted

into the database or the modelling layer; SQL views and Python outputs converge only at the Power BI data model, ensuring every dashboard figure has a traceable computational origin.

13.3 Data Pipeline Flow

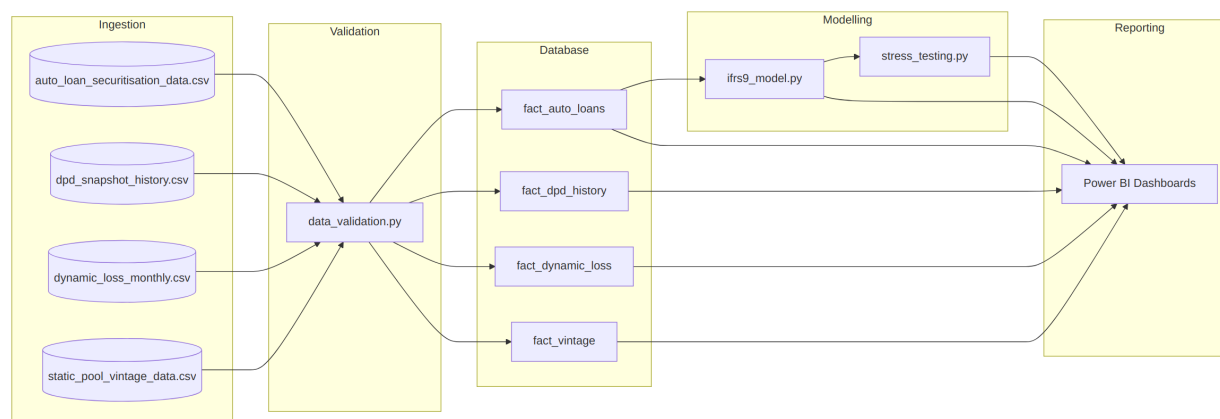


Figure 2: Diagram 2

Fig. 2 — Data Pipeline Flow. Every dataset passes through the same validation gate before reaching the database, and every model output (ECL, stress ECL) flows into the same Power BI semantic model as the raw SQL tables — there is only one reporting layer, not several disconnected ones.

13.4 Component Sequence — Monthly Processing Cycle

Fig. 4 — Python Analytics Layer Sequence Diagram. The sequence reflects a realistic monthly close: validation gates the database load, IFRS 9 output feeds the stress engine, and Power BI is the terminal consumer of every upstream computation.

13.5 Component Responsibilities

Layer	Component	Responsibility
Ingestion	CSV Extracts	Simulated servicer/core-banking data extracts
Validation	data_validation.py	Gatekeeper — blocks bad data from reaching the model or database
Storage	MySQL (securitisation_risk_analytics)	Normalized system of record for all portfolio, delinquency, loss, and vintage data
Logic	SQL Views & Stored Procedures	Reusable, centrally governed aggregation logic
Modelling	ifrs9_model.py	Loan-level and portfolio-level ECL computation
Modelling	stress_testing.py	Scenario-based ECL re-pricing

Layer	Component	Responsibility
Presentation	Power BI	Stakeholder-specific dashboards over a single semantic model

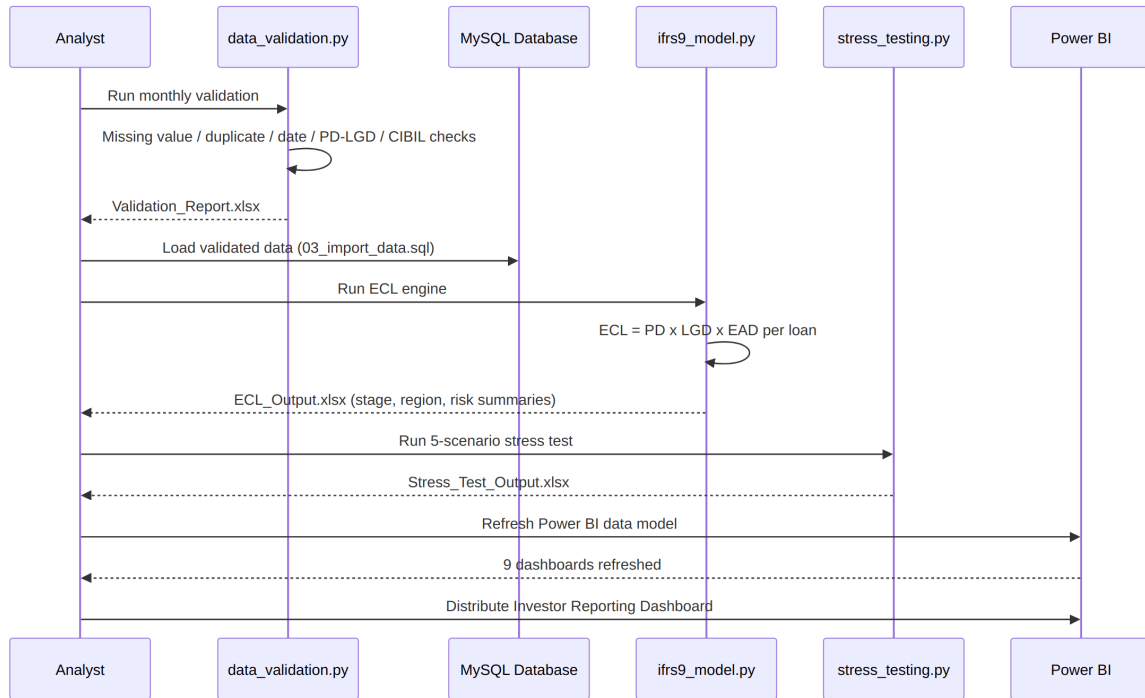


Figure 3: Diagram 3

14. Dataset Description

14.1 Overview

Table 1 — Dataset Overview Summary

Dataset	File	Grain	Destination Table
Auto Loans	auto_loan_securedat	One row per loan	fact_auto_loans
DPD History	dpd_snapshot_history	One row per loan per monthly snapshot	fact_dpd_history
Dynamic Loss	dynamic_loss_monthly	One row per reporting month (pool level)	fact_dynamic_loss
Vintage	static_pool_vintage_data	One row per vintage cohort per month-on-book	fact_vintage

14.2 Auto Loans Dataset

Purpose. This is the core loan-level fact table underpinning nearly every dashboard and every model computation in the platform. It carries origination, borrower, collateral, credit-risk, and IFRS 9 attributes for every loan in the securitised pool.

Business Meaning. Each row represents the full current-state profile of a single securitised auto loan — who the servicer is, what the loan terms are, the borrower’s current credit standing, the vehicle’s current value, and the IFRS 9 stage and provisioning figures currently held against it. This is the dataset the ECL engine and stress testing engine operate on directly.

Table 2 — fact_auto_loans Data Dictionary

Column	Type	Business Meaning
LoanID	VARCHAR(100), PK	Unique loan identifier
PoolID	VARCHAR(100)	Securitisation pool the loan belongs to
BorrowerID	VARCHAR(100)	Unique borrower identifier
ServicerID / ServicerName	VARCHAR	Entity servicing collections on this loan
OriginationDate	DATE	Date loan was originated
CutoffDate	DATE	Pool cutoff date at securitisation
MaturityDate	DATE	Scheduled final payment date
OriginalLoanAmount	DECIMAL(18,2)	Loan amount at origination
CurrentBalance	DECIMAL(18,2)	Outstanding principal balance
MonthlyEMI	DECIMAL(18,2)	Scheduled monthly instalment
InterestRate	DECIMAL(8,4)	Contractual interest rate
LTV_Current	DECIMAL(8,4)	Current loan-to-value ratio
DTI_Ratio	DECIMAL(8,4)	Borrower debt-to-income ratio
CIBIL_Score_Current	INT	Current bureau credit score (300-900)
DelinquencyDays	INT	Days past due as of latest snapshot
DelinquencyStatus	VARCHAR(50)	Current / 1-29 DPD / ... / Repossessed
IFRS9_Stage	VARCHAR(20)	IFRS 9 stage (1, 2, or 3)
PD_Estimate	DECIMAL(10,6)	Probability of Default (0-1)
LGD_Estimate	DECIMAL(10,6)	Loss Given Default (0-1)
EAD	DECIMAL(18,2)	Exposure at Default
ECL_Provision	DECIMAL(18,2)	Currently held ECL provision
RecoveryRate	DECIMAL(8,4)	Expected recovery rate on default
CurrentVehicleValue / CurrentPropertyValue	DECIMAL(18,2)	Current market value of collateral
AnnualIncome_INR	DECIMAL(18,2)	Borrower annual income
EmploymentType	VARCHAR(100)	Salaried / Self-Employed / etc.
Region / State / City	VARCHAR	Geographic attributes
VehicleType / VehicleAge / VehicleManufacturer / FuelType	VARCHAR/INT	Collateral attributes
LoanStatus	VARCHAR(50)	Active / Closed / Repossessed / Default

14.3 DPD History Dataset

Purpose. Captures a monthly delinquency snapshot for every loan, enabling roll-rate (transition matrix) analysis — the single most powerful early-warning tool in delinquency management, because it measures the *probability of movement* between buckets rather than just a

point-in-time count.

Business Meaning. Each row is one loan’s DPD status at one point in time, together with its *prior* bucket, so that transitions (e.g. 30–59 DPD → 60–89 DPD, or 60–89 DPD → Current, i.e. a “cure”) can be directly computed.

Table 3 — fact_dpd_history Data Dictionary

Column	Type	Business Meaning
SnapshotID	INT, PK, AUTO_INCREMENT	Surrogate key
LoanID	VARCHAR(100), FK → fact_auto_loans	Loan reference
SnapshotDate	DATE	Date of the delinquency snapshot
DPD_Days	INT	Days past due at this snapshot
DPD_Bucket	VARCHAR(50)	Current period’s bucket
DPD_Bucket_Prior	VARCHAR(50)	Prior period’s bucket (roll-rate source)
TransitionType	VARCHAR(100)	Descriptive transition label (e.g. “Cure”, “Roll Forward”)
CurrentBalance	DECIMAL(18,2)	Balance at time of snapshot

14.4 Dynamic Loss Dataset

Purpose. Tracks pool-level monthly cash flow and loss performance — the dataset that drives the Dynamic Loss and Waterfall dashboards and underlies the CPR (Conditional Prepayment Rate) and CDR (Conditional Default Rate) metrics investors watch most closely.

Business Meaning. Each row is one calendar month’s aggregate pool performance: total collections, recoveries, gross and net losses, collection efficiency, prepayment and default rates (annualised), excess spread, scheduled amortisation, and the end-of-period pool balance.

Table 4 — fact_dynamic_loss Data Dictionary

Column	Type	Business Meaning
ReportingDate	DATE, PK	Reporting month
CollectionsTotal	DECIMAL(18,2)	Total cash collected in the month
Recoveries_ThisMonth	DECIMAL(18,2)	Recoveries from defaulted/repossessed loans
GrossLoss_ThisMonth	DECIMAL(18,2)	Gross charge-offs before recoveries
NetLoss_ThisMonth	DECIMAL(18,2)	Gross loss less recoveries
CollectionEfficiency	DECIMAL(8,4)	Collections as a % of amounts due
CPR_Annualised	DECIMAL(8,4)	Annualised Conditional Prepayment Rate
CDR_Annualised	DECIMAL(8,4)	Annualised Conditional Default Rate
ExcessSpread_Monthly	DECIMAL(8,4)	Pool yield less funding cost and losses
ScheduledAmort	DECIMAL(18,2)	Scheduled principal amortisation

Column	Type	Business Meaning
EOP_Balance	DECIMAL(18,2)	End-of-period pool balance

14.5 Vintage Dataset

Purpose. Enables static pool vintage analysis — benchmarking loss and default performance of loans by their *origination cohort* as they age, which is the standard technique for detecting underwriting quality drift across origination periods.

Business Meaning. Each row represents one origination vintage (e.g. “2021-Q1”) at a specific number of months on book, tracking how that cohort’s pool factor, cumulative default rate, and cumulative net loss rate evolve over its life.

Table 5 — fact_vintage Data Dictionary

Column	Type	Business Meaning
VintageID	VARCHAR(100), PK (composite)	Origination cohort identifier (e.g. 2021-Q1)
MonthsOnBook	INT, PK (composite)	Age of the cohort in months
PoolFactor	DECIMAL(10,6)	Remaining balance / original balance
RemainingPoolBalance	DECIMAL(18,2)	Outstanding balance of the cohort
CumulativeNetLossRate	DECIMAL(10,6)	Cumulative net losses / original balance
CumulativeDefaultRate	DECIMAL(10,6)	Cumulative defaults / original balance

14.6 Relationships Across Datasets

fact_auto_loans is the central fact table: fact_dpd_history relates to it one-to-many via LoanID (many monthly snapshots per loan). fact_dynamic_loss and fact_vintage are pool-level and cohort-level aggregates respectively, and while they do not carry a direct foreign key to fact_auto_loans in this schema (by design — they are pre-aggregated reporting tables, consistent with how servicers typically deliver these monthly extracts), they are conceptually derived from the same loan population and are joined at the Power BI semantic layer through shared date and cohort dimensions.

15. Database Design

15.1 Normalization Approach

The schema follows a fact-table-centric design deliberately kept in Third Normal Form (3NF) for the transactional attributes while remaining pragmatic about pre-aggregated reporting tables (`fact_dynamic_loss`, `fact_vintage`), which mirror how these figures actually arrive from servicers and are not decomposed further — decomposing a monthly pool-level loss figure into a “more normalized” form would add joins with no analytical benefit. `fact_auto_loans` carries no repeating groups and every non-key attribute depends only on the primary key (`LoanID`), satisfying 1NF, 2NF, and 3NF. `fact_dpd_history` uses a surrogate `SnapshotID` primary key specifically because the natural key (`LoanID` + `SnapshotDate`) would be usable but a surrogate key simplifies downstream joins and is standard practice for high-volume snapshot/history tables.

15.2 Keys, Relationships, and Constraints

- **Primary Keys:** `fact_auto_loans.LoanID`, `fact_dpd_history.SnapshotID` (surrogate), `fact_dynamic_loss.ReportingDate`, `fact_vintage.(VintageID, MonthsOnBook)` composite.
- **Foreign Keys:** `fact_dpd_history.LoanID` references `fact_auto_loans.LoanID`, enforcing referential integrity so that no delinquency snapshot can exist for a loan not present in the core loan table.
- **Composite Key Rationale:** `fact_vintage` uses a composite primary key because neither `VintageID` nor `MonthsOnBook` alone is unique — the grain of the table is one row per cohort per age-in-months, and the composite key correctly enforces that grain at the database level.

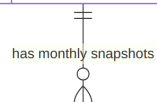
15.3 Entity-Relationship Diagram

Fig. 3 — Entity-Relationship Diagram (Core Schema). `fact_dpd_history` is the only table with an enforced foreign key relationship to `fact_auto_loans`; `fact_dynamic_loss` and `fact_vintage` are pool/cohort-level reporting tables joined at the semantic layer rather than the relational layer.

15.4 Indexing Strategy

Beyond the primary and foreign keys, seven additional indexes were created (see `02_create_tables.sql`, Section 16.2) covering the columns most frequently used in `WHERE` and `GROUP BY` clauses across the views and Power BI queries: `PoolID`, `ServicerName`, `IFRS9_Stage`, and `DelinquencyStatus` on `fact_auto_loans`; `SnapshotDate` on `fact_dpd_history`; `ReportingDate` on `fact_dynamic_loss`; and `VintageID` on `fact_vintage`. This mirrors standard OLAP-adjacent indexing practice: index the columns that dashboards slice and filter by, not necessarily every column.

fact_auto_loans		
varchar	LoanID	PK
varchar	PoolID	
varchar	BorrowerID	
varchar	ServicerName	
date	OriginationDate	
date	CutoffDate	
date	MaturityDate	
decimal	OriginalLoanAmount	
decimal	CurrentBalance	
decimal	InterestRate	
decimal	LTV_Current	
int	CIBIL_Score_Current	
int	DelinquencyDays	
varchar	DelinquencyStatus	
varchar	IFRS9_Stage	
decimal	PD_Estimate	
decimal	LGD_Estimate	
decimal	EAD	
decimal	ECL_Provision	
varchar	Region	
varchar	State	



fact_dpd_history		
int	SnapshotID	PK
varchar	LoanID	FK
date	SnapshotDate	
int	DPD_Days	
varchar	DPD_Bucket	
varchar	DPD_Bucket_Prior	
varchar	TransitionType	
decimal	CurrentBalance	

fact_dynamic_loss		
date	ReportingDate	PK
decimal	CollectionsTotal	
decimal	Recoveries_ThisMonth	
decimal	GrossLoss_ThisMonth	
decimal	NetLoss_ThisMonth	
decimal	CollectionEfficiency	
decimal	CPR_Annualised	
decimal	CDR_Annualised	
decimal	ExcessSpread_Monthly	
decimal	ScheduledAmort	
decimal	EOP_Balance	

fact_vintage		
varchar	VintageID	PK
int	MonthsOnBook	PK
decimal	PoolFactor	
decimal	RemainingPoolBalance	
decimal	CumulativeNetLossRate	
decimal	CumulativeDefaultRate	

Figure 4: Diagram 4

16. SQL Implementation

The database layer is delivered as five sequential, idempotent scripts, each with a single clear responsibility — a structure chosen deliberately so the entire schema can be torn down and rebuilt deterministically, which is essential for both local development and CI-style automated testing of the pipeline.

16.1 01_create_database.sql

Purpose. Creates the `securitisation_risk_analytics` database with a UTF-8MB4 character set, guaranteeing full Unicode support (important for city/state name fields) and consistent collation behaviour for string comparisons and sorting.

Business Logic. The script is destructive-safe by design — it drops the database if it already exists before recreating it, so re-running the setup never leaves the schema in an inconsistent half-migrated state.

Workflow. Drop database if exists → create database with `utf8mb4/utf8mb4_unicode_ci` → switch context with `USE` → verify with `SELECT DATABASE()`.

Expected Output. A single-row confirmation that `securitisation_risk_analytics` is the active database.

Advantages / Best Practices. Using `utf8mb4` rather than the legacy `utf8` alias avoids MySQL's well-known 3-byte Unicode truncation issue; the explicit `DROP DATABASE IF EXISTS` pattern is standard for reproducible local/dev environments (never used against a production database in practice, but appropriate here).

16.2 02_create_tables.sql

Purpose. Defines all four fact tables, their column-level constraints, foreign key relationships, and supporting indexes.

Business Logic. Data types are chosen to reflect real banking precision requirements: monetary columns use `DECIMAL(18,2)` rather than floating-point types to avoid rounding error accumulation across millions of aggregated rows — a non-negotiable practice in any financial computation context. Rate and ratio fields (`PD_Estimate`, `LGD_Estimate`, `InterestRate`, `LTV_Current`) use `DECIMAL` with extended precision (`DECIMAL(10,6)` for `PD/LGD`) because these values feed directly into the ECL formula, where even small rounding drift compounds across a large loan book.

Workflow. Drop-and-recreate each table in dependency order (`fact_auto_loans` first, since `fact_dpd_history` depends on it) → declare the foreign key constraint `fk_dpd_loan` → create all indexes → confirm with `SHOW TABLES`.

Expected Output. Four tables created successfully with the foreign key constraint active, followed by a table listing confirming `fact_auto_loans`, `fact_dpd_history`, `fact_dynamic_loss`, and `fact_vintage`.

Advantages / Best Practices. Explicit foreign key constraints (rather than relying on application-layer integrity alone) ensure the database itself rejects orphaned DPD snapshot rows; the composite primary key on `fact_vintage` correctly models the table's true grain rather than adding an unnecessary surrogate key.

16.3 03_import_data.sql

Purpose. Post-load verification script confirming that data has landed correctly in each of the four fact tables after the CSV-to-MySQL load step.

Business Logic. Rather than trusting the load process silently, this script gives an explicit row-count reconciliation across all four tables in a single query using UNION ALL, so a missing or partial load is immediately visible.

Workflow. One SELECT COUNT(*) per table, unioned into a single result set labelled by table name.

Expected Output. A four-row result set showing record counts for fact_auto_loans, fact_dpd_history, fact_dynamic_loss, and fact_vintage — the first checkpoint an analyst reviews after every monthly data refresh.

Advantages / Best Practices. This pattern — a single reconciliation query run immediately after every load — is standard operating discipline in any regulated data pipeline, and is deliberately kept separate from the table-creation script so it can be re-run independently after every monthly refresh without touching the schema.

16.4 04_views.sql

Purpose. Encapsulates the six most frequently needed aggregations directly at the database layer, so that Power BI, ad hoc SQL clients, and any future consuming application all draw from identical, centrally governed business logic rather than each re-implementing the same GROUP BY independently.

Business Logic and Coverage:

View	Business Logic
vw_portfolio_summary	Pool-level rollup — loan count, balance, ECL, average PD/LGD/rate, grouped by PoolID
vw_ifrs9_summary	IFRS 9 stage-wise rollup — the direct source of the IFRS-9 Dashboard's stage exposure cards
vw_dpd_summary	Delinquency bucket rollup — average DPD and balance by DPD_Bucket
vw_servicer_summary	Servicer-level performance and risk comparison
vw_dynamic_loss_summary	Pass-through of pool-level monthly loss/prepayment metrics
vw_vintage_summary	Pass-through of cohort-level vintage curve data

Workflow. Each view is dropped and recreated idempotently, then the script confirms creation via SHOW FULL TABLES WHERE Table_type='VIEW'.

Expected Output. Six views listed and immediately queryable, each returning a pre-aggregated result set ready for direct Power BI import or ad hoc analysis.

Advantages / Best Practices. Centralising aggregation logic in views rather than in DAX or application code means a single definitional change (e.g. redefining what counts as “high risk”) only needs to be made once, at the data layer, and every downstream consumer inherits it automatically — a core principle of governed enterprise reporting.

16.5 05_stored_procedures.sql

Purpose. Encapsulates parameterised and reusable query logic that goes beyond what a static view can express — specifically, scenario-driven stress testing directly at the database layer.

Business Logic and Coverage:

- `sp_GetPortfolioSummary()` — a zero-parameter procedure returning top-line portfolio KPIs (loan count, balance, total ECL, average PD/LGD), used as a lightweight callable alternative to querying `vw_portfolio_summary` directly.
- `sp_GetIFRS9Summary()` — returns the IFRS 9 stage-wise summary ordered by stage, functionally equivalent to `vw_ifrs9_summary` but demonstrating the procedural alternative pattern.
- `sp_GetStressScenario(PDMultiplier, LGDMultiplier)` — a genuinely parameterised procedure that computes loan-level stressed PD, stressed LGD, and stressed ECL on the fly for *any* multiplier combination passed in, without requiring a code change. This is the SQL-layer analogue of the scenario logic implemented independently in Python’s `stress_testing.py`, demonstrating that the same business rule ($\text{EAD} \times \text{Stressed PD} \times \text{Stressed LGD}$) is correctly and consistently implemented at both the database and application layers.

Workflow. Each procedure is defined between `DELIMITER $$` blocks to allow semicolons inside the procedure body, dropped and recreated idempotently, and callable via `CALL sp_GetStressScenario(1.5, 1.3);` for an ad hoc severe-recession-equivalent scenario.

Expected Output. Three callable stored procedures; `sp_GetStressScenario` returns a full loan-level result set with `StressPD`, `StressLGD`, and `StressECL` columns for the multipliers supplied.

Advantages / Best Practices. Exposing a parameterised stored procedure for stress scenarios means a risk analyst can test an arbitrary macro scenario directly from a SQL client or a reporting tool’s stored-procedure connector, without needing to run the full Python pipeline — a genuinely useful operational shortcut for ad hoc “what if PD rose by 40%?” questions during a risk committee meeting.

17. Python Analytics Layer

17.1 data_validation.py — Architecture and Purpose

This module is the platform’s data quality gate: no dataset is permitted to reach the IFRS 9 or stress testing engines, or the database load step, without first passing through it. It is structured around a set of small, single-purpose functions (`check_required_columns`, `check_missing_values`, `check_duplicate_loanid`, `validate_dates`, `check_negative_values`, `validate_pd_lgd`, `validate_cibil`, `validate_dpd`) that are each applied independently to all four datasets, with results written to a multi-sheet `Validation_Report.xlsx` workbook for full auditability.

Validation Rules Implemented:

- **Schema Validation** — `check_required_columns` confirms every dataset contains its full expected column set before any further processing; a missing column raises a hard `ValueError` rather than allowing silent downstream failure.
- **Missing Value Analysis** — computes both count and percentage of nulls per column, giving analysts an immediate sense of data completeness severity rather than a binary pass/fail.
- **Duplicate LoanID Detection** — flags any `LoanID` that appears more than once in the Auto Loans dataset, since a duplicate loan identifier would silently double-count exposure and balance in every downstream aggregation.
- **Date Validation** — dynamically detects every column with “Date” in its name and coerces it to a proper datetime, counting any value that fails conversion — catching malformed date strings that would otherwise pass through as nulls in the modelling layer.
- **Negative Value Checks** — scans all numeric columns for negative values, which are almost always a data quality defect for fields like balance, EAD, or loss amounts in this domain.
- **PD/LGD Range Validation** — confirms `PD_Estimate` and `LGD_Estimate` fall strictly within `[0, 1]`, since both are probabilities/rates by definition and any value outside this range would produce a nonsensical or even negative ECL.
- **CIBIL Score Validation** — confirms `CIBIL_Score_Current` falls within the valid Indian bureau score range of 300–900.
- **DPD Validation** — flags any negative `DPD_Days` value in the delinquency history dataset.

Output. A structured `Validation_Report.xlsx` with dedicated sheets per dataset per check type (e.g. `Auto_Missing`, `Auto_Duplicates`, `PD_LGD_Check`, `CIBIL_Check`), plus a consolidated `Validation_Summary` sheet giving a single-glance pass/fail overview across all four datasets — the exact artefact a data quality analyst would attach to a monthly close sign-off.

17.2 ifrs9_model.py — IFRS 9 Expected Credit Loss Engine

IFRS 9 in Detail. IFRS 9 replaced the prior “incurred loss” impairment model with a forward-looking “expected loss” model, requiring institutions to recognise credit losses before an actual default event occurs. The standard organises financial assets into three stages based on the extent of credit deterioration since origination:

- **Stage 1** — Performing assets with no significant increase in credit risk since origination. Institutions recognise 12-month expected credit losses.
- **Stage 2** — Assets that have experienced a significant increase in credit risk since origination but are not yet credit-impaired. Institutions recognise lifetime expected credit losses.
- **Stage 3** — Credit-impaired assets (effectively, defaulted). Lifetime expected credit losses are recognised, and interest income is calculated on the net carrying amount rather than gross exposure.

The Core Formula. $ECL = PD \times LGD \times EAD$, where:

- **PD (Probability of Default)** — the likelihood, expressed as a decimal between 0 and 1, that the borrower will default within the relevant time horizon (12-month for Stage 1, lifetime for Stage 2/3).
- **LGD (Loss Given Default)** — the proportion of exposure the institution expects to lose if default occurs, after accounting for collateral recovery (vehicle resale value in this context) — expressed as a decimal between 0 and 1.
- **EAD (Exposure at Default)** — the expected outstanding exposure at the point of default, which for an amortising auto loan is typically close to, but not always identical to, the current balance.

Module Workflow. The module loads `auto_loan_securitisation_data.csv`, validates the presence of all fourteen required columns, coerces PD/LGD/EAD/ECL_Provision to numeric with null-safe defaulting, then computes $Calculated_ECL = PD_Estimate \times LGD_Estimate \times EAD$ at loan level. It immediately compares this to the currently held ECL_Provision to produce a Provision_Variance and a Provision_Status flag (“Under Provisioned”, “Over Provisioned”, “Balanced”) — a critical reconciliation step, since the whole point of an independent ECL recalculation is to catch provisioning drift between what a servicing system holds and what the risk engine’s own model produces.

Risk Categorisation. Loans are bucketed into four risk categories purely on PD, using `numpy.select` against ordered thresholds:

Table 8 — IFRS 9 Risk Category Bands

Category	PD Range
Low	$PD < 2\%$
Moderate	$2\% \leq PD < 5\%$
High	$5\% \leq PD < 10\%$
Very High	$PD \geq 10\%$

Portfolio-Level Outputs. The module produces a Portfolio Summary (total loans, portfolio balance, total exposure, existing vs. calculated ECL, average PD/LGD/rate/LTV/CIBIL), an IFRS 9 Stage Summary (loan count, balance, exposure, existing and calculated ECL, average PD/LGD/CIBIL grouped by stage), Region and State summaries sorted by calculated ECL descending (surfacing geographic risk concentration immediately), a Risk Category summary, a Top-20 Highest-ECL loan list (the exact watchlist a credit risk officer would pull first), and a portfolio-level Coverage Ratio ($Calculated\ ECL / Portfolio\ Balance \times 100$) — the single metric most commonly quoted to a CRO or board risk committee as the headline provisioning adequacy indicator.

Output. `ECL_Output.xlsx` with eight sheets: Portfolio Summary, IFRS Stage Summary, Region Summary, State Summary, Risk Summary, Coverage Ratio, Top 20 High Risk Loans, and full Loan Level ECL detail — feeding directly into the IFRS-9, Portfolio Overview, and Executive Summary Power BI dashboards.

17.3 `stress_testing.py` — Macroeconomic Stress Testing Engine

Purpose. Re-prices the entire loan portfolio’s ECL under a defined set of macroeconomic stress scenarios, answering the question every risk committee and regulator asks: “*What happens to our provisioning if conditions deteriorate?*”

Formula. $Stress\ ECL = EAD \times (PD \times PD\ Multiplier) \times (LGD \times LGD\ Multiplier)$, with both stressed PD and stressed LGD capped at a ceiling of 1.0 (`clip(upper=1)`) since neither can logically exceed 100%.

Table 9 — Stress Testing Scenario Calibration

Scenario	PD Multiplier	LGD Multiplier	Interpretation
Base	1.00x	1.00x	Current-state, unstressed ECL
Mild Recession	1.25x	1.10x	Moderate unemployment uptick, modest collateral value decline
Severe Recession	1.50x	1.30x	Sharp unemployment rise, meaningful used-vehicle value decline
Financial Crisis	2.00x	1.50x	2008-style systemic credit event and collateral market collapse
Pandemic Shock	2.25x	1.60x	Sudden-stop income shock combined with severely depressed resale markets

The calibration logic reflects a well-established pattern in credit stress testing: PD is more sensitive to macro stress than LGD, because default frequency responds directly and quickly to unemployment and income shocks, while LGD (loss severity given default) is bounded by collateral value, which — even in a severe downturn — rarely falls to zero. This is why every scenario’s PD multiplier is set higher than its corresponding LGD multiplier.

Workflow. For each of the five scenarios, the script computes `Stress_PD`, `Stress_LGD`, `Stress_ECL`, and `ECL_Change` (stress ECL less currently held provision) at loan level, then rolls this up into an IFRS-9-stage-wise summary sheet named after the scenario. For the Financial Crisis scenario specifically, a Top-20 highest-stress-ECL loan list is additionally extracted — the scenario most likely to be the one a regulator or board asks to see loan-level detail for. A final `Scenario Summary` sheet consolidates portfolio balance, exposure, existing ECL, stress ECL, and the ECL increase for all five scenarios side by side.

Business Interpretation. The scenario comparison table is the platform’s single most decision-relevant output: it directly quantifies additional capital an institution would need to hold under each stress scenario, which is precisely the analysis feeding both internal capital adequacy assessment and, in many jurisdictions, formal regulatory stress test submissions.

Fig. 6 — Stress Testing Scenario Comparison Flow. All five scenarios are computed independently against the same base portfolio, then consolidated into a single comparison sheet that becomes the direct source of the Stress Testing Dashboard’s scenario comparison table.

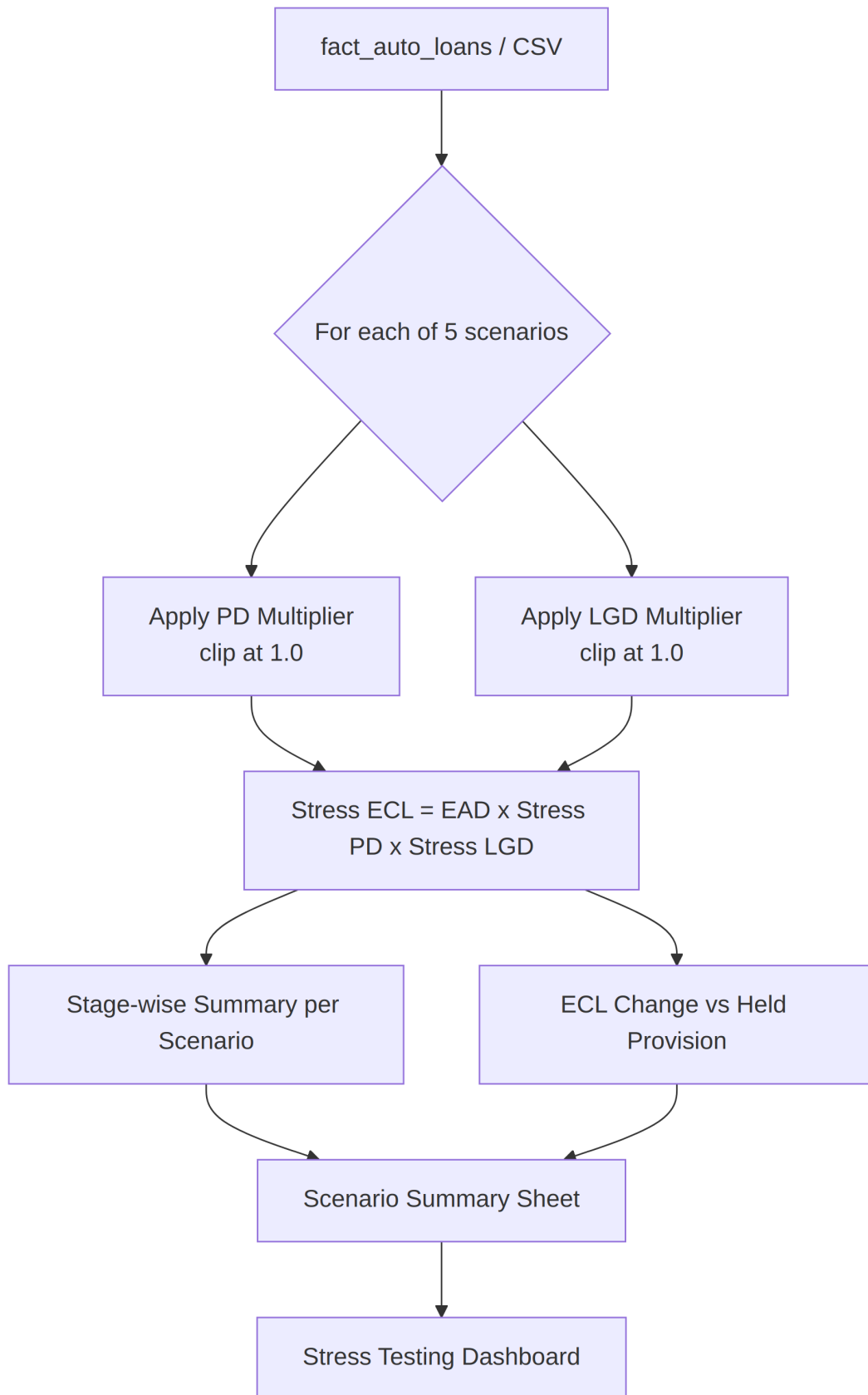


Figure 5: Diagram 5

18. Power BI Implementation

Nine dashboards sit on a single shared semantic model, meaning a slicer selection for IFRS9_Stage, DelinquencyStatus, or ServicerName behaves consistently across every page — a deliberate design choice reflecting how enterprise BI governance works: one model, many purpose-built views, never a divergent metric definition per page.

Table 10 — Dashboard-to-Stakeholder Mapping

Dashboard	Primary Stakeholder
Executive Summary	CRO / Executive Committee
Portfolio Overview	Portfolio Managers
IFRS-9	Credit Risk / Finance (Provisioning)
DPD & Roll Rate	Collections / Servicing Management
Vintage	Underwriting / Credit Policy
Dynamic Loss	Structured Finance / Treasury
Waterfall	Trustee / Structured Finance
Stress Testing	Risk Committee / Regulator
Investor Reporting	ABS Investors / Trustee

18.1 Executive Summary Dashboard

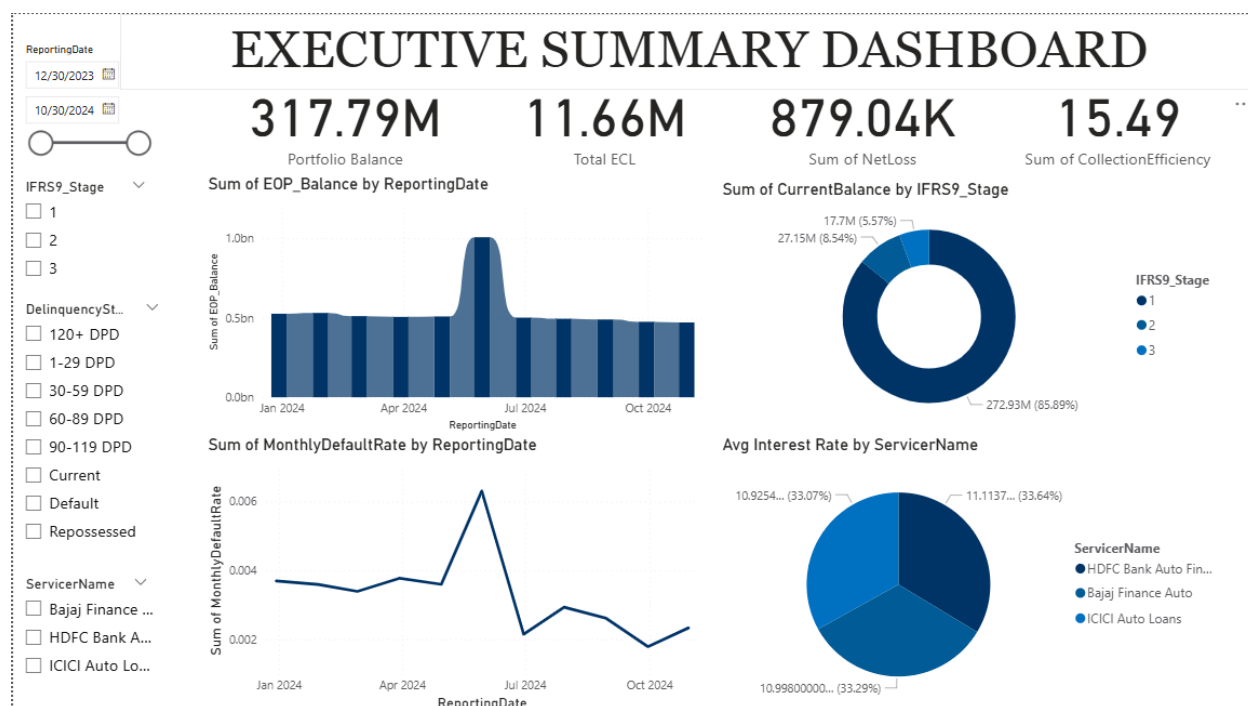


Figure 6: Executive Summary Dashboard

Fig. 10 — Executive Summary Dashboard.

Purpose. The single-page view a CRO or executive committee member opens first — top-line portfolio health with no drill-down required.

KPIs. Portfolio Balance (317.79M), Total ECL (11.66M), Sum of Net Loss (879.04K), Sum of Collection Efficiency (15.49).

Visuals. EOP Balance trend by ReportingDate (bar chart showing pool amortisation over time, with a visible mid-year spike corresponding to a portfolio replenishment or reporting anomaly worth footnoting); Current Balance by IFRS9 Stage (donut, showing 85.89% of balance in Stage 1); Monthly Default Rate trend line; Average Interest Rate by Servicer (pie).

Slicers. ReportingDate (range), IFRS9_Stage, DelinquencyStatus, ServicerName.

Business Insights. The IFRS9 Stage donut immediately quantifies credit migration risk — with 85.89% of balance still in Stage 1, the pool is fundamentally performing, but the combined 14.11% in Stage 2 and Stage 3 is the portion driving nearly all of the 11.66M ECL, since Stage 2/3 provisioning is materially higher per unit of exposure than Stage 1.

DAX Measures (representative).

```
Total ECL = SUM(fact_auto_loans[ECL_Provision])
Portfolio Balance = SUM(fact_auto_loans[CurrentBalance])
Stage 1 Balance % =
    DIVIDE(
        CALCULATE(SUM(fact_auto_loans[CurrentBalance]), fact_auto_loans[IFRS9_Stage] = "1"),
        [Portfolio Balance]
    )
Monthly Default Rate =
    DIVIDE(
        CALCULATE(DISTINCTCOUNT(fact_auto_loans[LoanID]), fact_auto_loans[DelinquencyStatus] =
            "Default"),
        DISTINCTCOUNT(fact_auto_loans[LoanID])
    )
```

Business Decisions Supported. Board-level risk appetite discussions; go/no-go on new pool origination volume; early flag for provisioning trend requiring finance team attention.

18.2 Portfolio Overview Dashboard

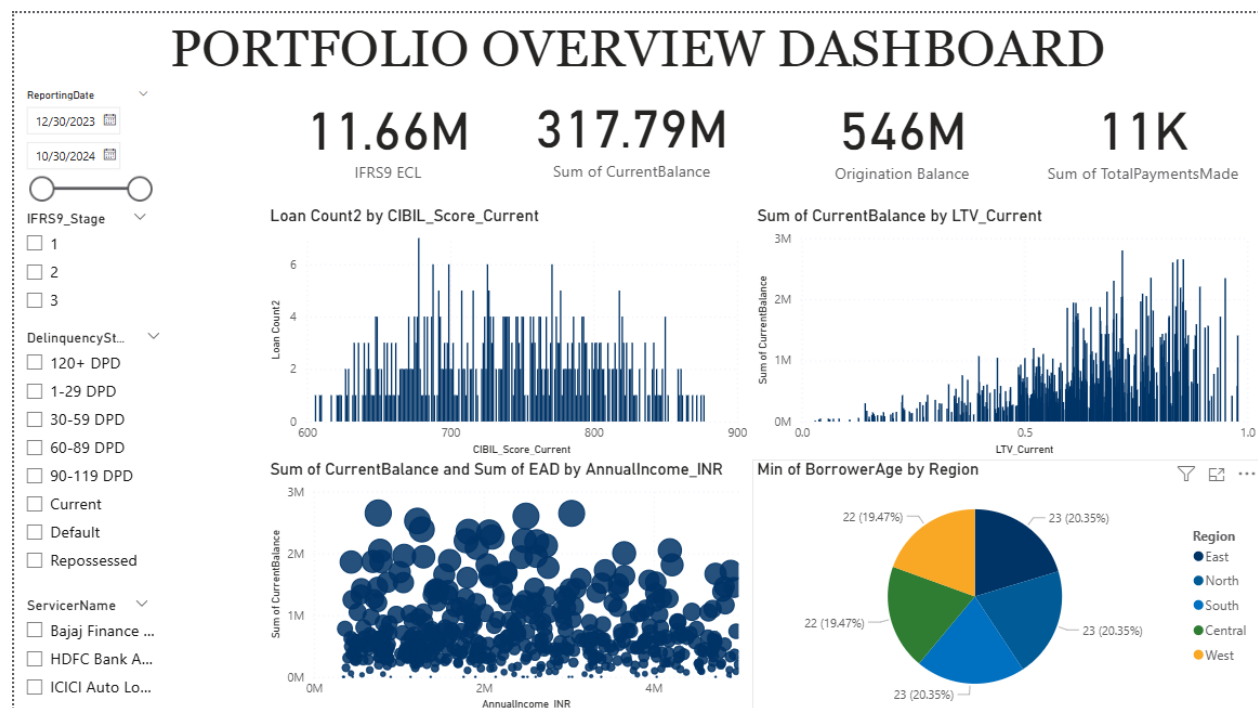


Figure 7: Portfolio Overview Dashboard

Fig. 11 — Portfolio Overview Dashboard.

Purpose. Detailed portfolio composition view for portfolio managers — credit quality distribution, LTV distribution, and geographic concentration.

KPIs. IFRS9 ECL (11.66M), Sum of Current Balance (317.79M), Origination Balance (546M), Sum of Total Payments Made (11K).

Visuals. Loan Count by CIBIL Score histogram (credit quality distribution across the book); Current Balance by LTV_Current histogram (collateral risk distribution, right-skewed toward higher LTV — a leading indicator of loss severity risk); Current Balance and EAD bubble scatter by Annual Income (income-vs-exposure concentration view); Minimum Borrower Age by Region pie.

Slicers. ReportingDate, IFRS9_Stage, DelinquencyStatus, ServicerName.

Business Insights. The origination-to-current-balance ratio ($317.79\text{M} / 546\text{M} \approx 58\%$) indicates the pool has amortised down to roughly 58% of its original size — a pool factor consistent with a maturing securitisation partway through its expected life, directly relevant to when the deal is likely to reach its clean-up call threshold.

DAX Measures (representative).

```
Pool Factor = DIVIDE([Portfolio Balance], SUM(fact_auto_loans[OriginalLoanAmount]))
Avg LTV = AVERAGE(fact_auto_loans[LTV_Current])
High LTV Exposure = CALCULATE([Portfolio Balance], fact_auto_loans[LTV_Current] > 0.8)
```

Business Decisions Supported. Underwriting policy calibration (LTV caps, CIBIL cutoffs); geographic origination strategy; concentration risk limits.

18.3 IFRS-9 Dashboard

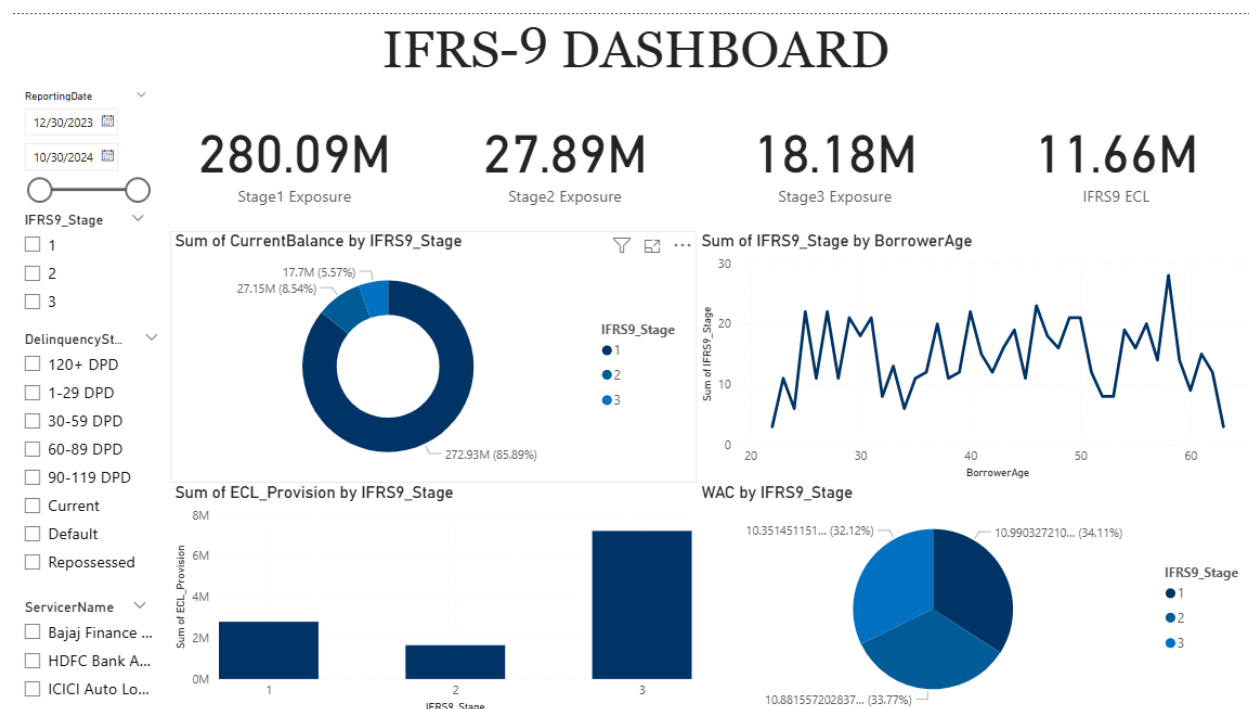


Figure 8: IFRS-9 Dashboard

Fig. 12 — IFRS-9 Dashboard.

Purpose. The dedicated provisioning dashboard for credit risk and finance teams — this is the page that would be screenshotted directly into a month-end provisioning memo.

KPIs. Stage 1 Exposure (280.09M), Stage 2 Exposure (27.89M), Stage 3 Exposure (18.18M), IFRS9 ECL (11.66M).

Visuals. Current Balance by IFRS9 Stage (donut); IFRS9 Stage by Borrower Age (trend showing no strong age-driven staging pattern, i.e. staging is being driven by behavioural/credit factors rather than demographic ones — an important model-validation observation); ECL Provision by IFRS9 Stage (bar, showing the largest single provision sits against Stage 3 despite it having the smallest exposure — as expected, since Stage 3 carries by far the highest per-unit PD $\times$ LGD); Weighted Average Coupon (WAC) by IFRS9 Stage (pie, roughly even split — indicating pricing is not yet meaningfully differentiated by realised credit migration).

Slicers. ReportingDate, IFRS9_Stage, DelinquencyStatus, ServicerName.

Business Insights. Stage 3 represents only about 5.6% of exposure (18.18M of 326M total) yet the ECL Provision bar chart shows it carries the largest single provisioning bucket — a textbook illustration of why stage migration, not raw exposure, is the correct lens for provisioning risk.

DAX Measures (representative).

```
Stage 3 Exposure = CALCULATE(SUM(fact_auto_loans[EAD]), fact_auto_loans[IFRS9_Stage] = "3")
Coverage Ratio % = DIVIDE([Total ECL], [Portfolio Balance])
WAC = DIVIDE(
    SUMX(fact_auto_loans, fact_auto_loans[InterestRate] * fact_auto_loans[CurrentBalance]),
    [Portfolio Balance]
)
```

Business Decisions Supported. Quarterly provisioning sign-off; model validation review (does staging correlate with the risk drivers it should?); auditor and regulator walkthrough support.

18.4 DPD & Roll Rate Dashboard

Fig. 13 — DPD & Roll Rate Dashboard.

Purpose. The primary early-warning tool for collections and servicing management — the roll-rate matrix quantifies the probability of a loan moving from one delinquency bucket to another between consecutive snapshots.

KPIs. 30+ DPD Exposure (44.85M), 60+ DPD Exposure (33.64M), 90+ DPD Exposure (17.70M).

Visuals. The centrepiece is a colour-coded roll-rate transition matrix (prior bucket in rows, current bucket in columns): reading the “1-29 DPD” row shows 48% roll forward to 30-59 DPD while 89% of that same starting bucket (in a different column read) actually cures back to Current — illustrating that early-stage delinquency has a meaningfully high cure rate, a critical input to LGD calibration. The “90-119 DPD” row shows a 95% roll rate into Default, meaning once a loan reaches 90-119 days past due it is a near-certain loss event, with only a 5% repossession-recovery path and negligible cure probability. Below the matrix, Sum of ConsecutiveMonthsDelinquent trend by SnapshotDate (line, showing a clear upward trend through 2024) and Loan Count by DPD_Bucket (a steep, expected drop-off curve from Current down to Default).

Slicers. ReportingDate, IFRS9_Stage, DelinquencyStatus, ServicerName.

DPD & ROLL RATE DASHBOARD

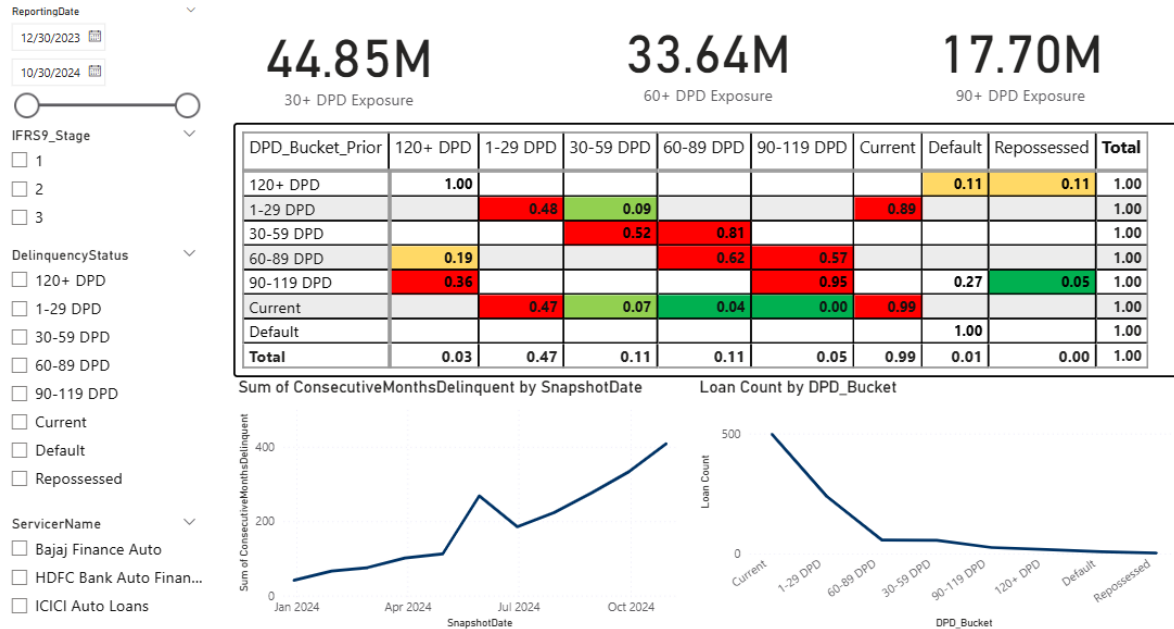


Figure 9: DPD and Roll Rate Dashboard

Business Insights. The roll-rate matrix is the platform’s most operationally actionable visual: it directly tells a collections manager where to focus resources — the 1-29 DPD bucket has the highest cure probability and therefore the highest return on early intervention effort, while 90+ DPD accounts are a near-certain loss and are better routed straight to recovery/legal rather than continued soft collections.

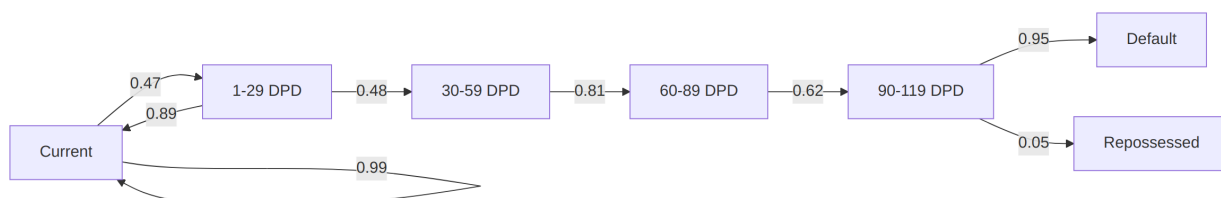


Figure 10: Diagram 6

Fig. 8 — DPD Roll Rate Transition Model (illustrative subset of the full matrix). DAX Measures (representative).

```

Roll Rate % =
    DIVIDE(
        CALCULATE(DISTINCTCOUNT(fact_dpd_history[LoanID]),
            fact_dpd_history[DPD_Bucket_Prior] = SELECTEDVALUE(fact_dpd_history[DPD_Bucket_Prior])),
        CALCULATE(DISTINCTCOUNT(fact_dpd_history[LoanID]),
            ALLEXCEPT(fact_dpd_history, fact_dpd_history[DPD_Bucket_Prior]))
    )
30+ DPD Exposure = CALCULATE([Portfolio Balance], fact_auto_loans[DelinquencyDays] >= 30)
  
```

Business Decisions Supported. Collections resource allocation by bucket; early-

intervention program design; LGD model calibration (cure rates feed directly into loss severity assumptions).

18.5 Vintage Dashboard

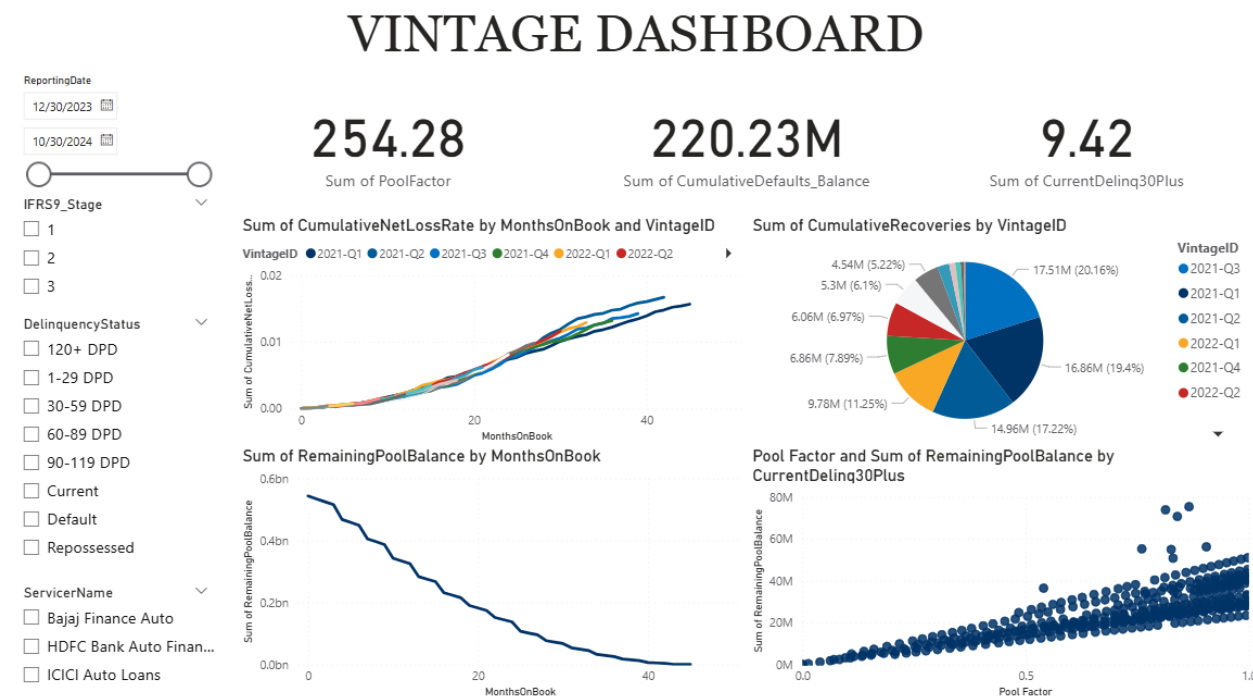


Figure 11: Vintage Dashboard

Fig. 14 — Vintage Dashboard.

Purpose. Static pool vintage analysis for underwriting and credit policy — benchmarking origination cohorts against each other as they age, to detect underwriting quality drift.

KPIs. Sum of Pool Factor (254.28), Sum of Cumulative Defaults Balance (220.23M), Sum of Current Delinquency 30+ (9.42).

Visuals. Cumulative Net Loss Rate by Months on Book, one line per vintage cohort (2021-Q1 through 2022-Q2) — the standard vintage curve chart, where a cohort's line sitting persistently above its peers at the same months-on-book is the classic signature of underwriting quality deterioration in that origination period; Cumulative Recoveries by VintageID (pie); Remaining Pool Balance by Months on Book (declining curve as the pool amortises); Pool Factor vs Remaining Pool Balance scatter by Current 30+ Delinquency.

Slicers. ReportingDate, IFRS9_Stage, DelinquencyStatus, ServicerName.

Business Insights. With six vintage cohorts plotted together, any cohort whose cumulative net loss rate curve breaks away from the cluster at the same months-on-book is immediately visible — this is precisely how a credit policy team would identify, for example, that loans originated in a specific quarter under a since-tightened underwriting policy are underperforming their peers, informing whether that policy change should be made permanent or reversed.

DAX Measures (representative).

Cumulative Net Loss Rate = DIVIDE(SUM(fact_vintage[RemainingPoolBalance]) - [Original Vintage B
Vintage Pool Factor = AVERAGE(fact_vintage[PoolFactor])

Business Decisions Supported. Underwriting policy retrospectives; identifying which origination cohorts to exclude from future securitisation pools; credit policy committee reporting.

18.6 Dynamic Loss Dashboard

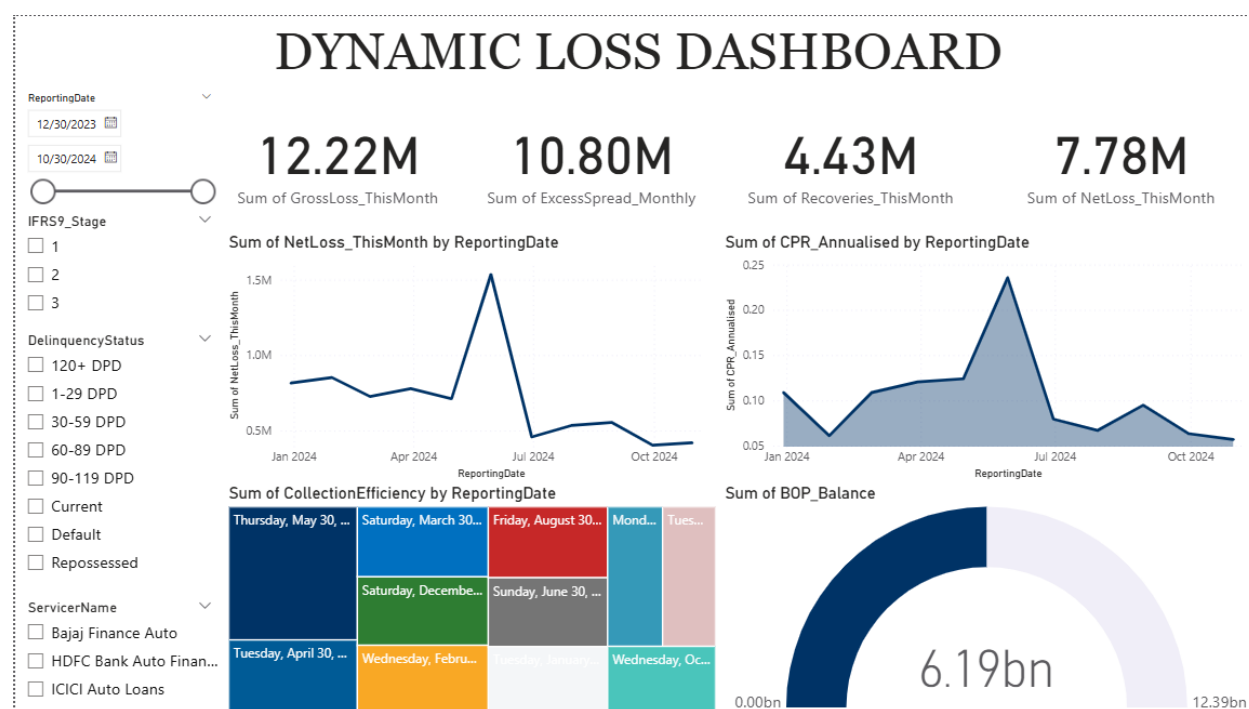


Figure 12: Dynamic Loss Dashboard

Fig. 15 — Dynamic Loss Dashboard.

Purpose. Month-by-month pool performance tracking for structured finance and treasury — the dashboard most directly tied to whether a deal’s performance triggers (early amortisation, reserve fund top-up) are at risk of breach.

KPIs. Sum of Gross Loss This Month (12.22M), Sum of Excess Spread Monthly (10.80M), Sum of Recoveries This Month (4.43M), Sum of Net Loss This Month (7.78M).

Visuals. Net Loss by Reporting Date (line, showing a sharp mid-2024 spike from roughly 0.7M to over 1.5M before reverting) — precisely the kind of anomaly that would trigger a trustee inquiry and a root-cause investigation into whether it reflects a genuine credit event or a one-off large repossession settlement; CPR Annualised trend (area chart, tracking the same mid-year spike pattern, suggesting the anomaly may be prepayment-related rather than purely credit-related); Collection Efficiency by Reporting Date (calendar-style heat table); BOP Balance gauge (6.19bn against a 12.39bn range).

Slicers. ReportingDate, IFRS9_Stage, DelinquencyStatus, ServicerName.

Business Insights. The fact that both Net Loss and CPR spike in the same reporting period is a genuinely useful diagnostic: a pure credit deterioration event would move loss metrics without necessarily moving prepayment behaviour, whereas a correlated spike in both is more consistent with either a data artefact (worth validating against the `data_validation.py` output for that period) or a broader macro/liquidity event affecting the whole borrower base simultaneously.

DAX Measures (representative).

Net Loss Rate = $\text{DIVIDE}(\text{SUM}(\text{fact_dynamic_loss}[\text{NetLoss_ThisMonth}]), \text{SUM}(\text{fact_dynamic_loss}[\text{EOP_Bal}])$
 Excess Spread = $\text{AVERAGE}(\text{fact_dynamic_loss}[\text{ExcessSpread_Monthly}])$

Business Decisions Supported. Early-amortisation trigger monitoring; reserve fund adequacy assessment; anomaly investigation trigger for the servicer relationship team.

18.7 Waterfall Dashboard

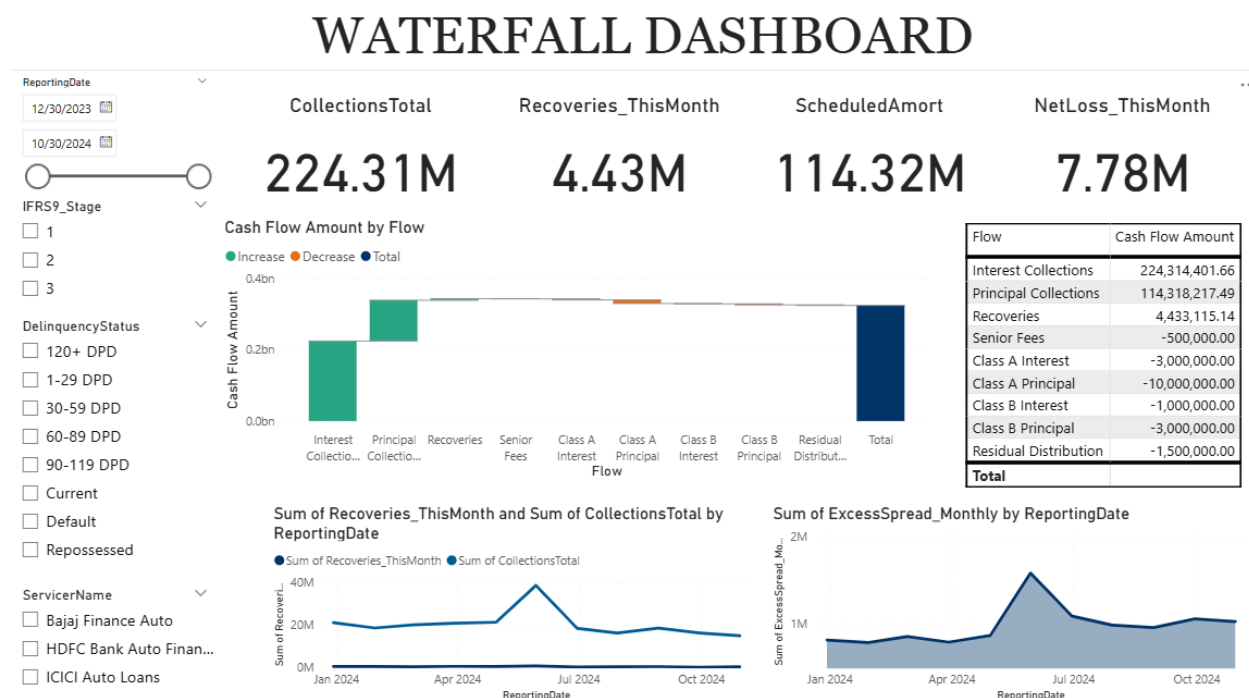


Figure 13: Waterfall Dashboard

Fig. 16 — Waterfall Dashboard.

Purpose. Replicates the monthly cash-flow waterfall allocation for trustee and structured finance reporting — showing exactly how collected cash was distributed down the payment priority.

KPIs. Collections Total (224.31M), Recoveries This Month (4.43M), Scheduled Amortisation (114.32M), Net Loss This Month (7.78M).

Visuals. A waterfall chart moving from Interest Collections and Principal Collections (increases) through Recoveries, Senior Fees, Class A Interest, Class A Principal, Class B Interest, Class B Principal, and Residual Distribution (decreases) down to a Total bar — alongside a detailed cash-flow table breaking out every line item's exact amount (e.g. Class A Principal: -10,000,000; Class B Principal: -3,000,000; Residual Distribution: -1,500,000); supporting trend lines for Recoveries and Collections Total, and Excess Spread over time.

Slicers. ReportingDate, IFRS9_Stage, DelinquencyStatus, ServicerName.

Business Insights. The waterfall table makes the credit enhancement structure directly legible: Class A Principal (10M) is materially larger than Class B Principal (3M), consistent with Class A being the much larger senior tranche, and the residual distribution of 1.5M confirms cash is flowing all the way through to the equity/first-loss holder this period — meaning

realised losses have not yet been severe enough to interrupt the bottom of the waterfall, a reassuring signal for the deal's overall health.

DAX Measures (representative).

Total Waterfall Outflow = [Senior Fees] + [Class A Interest] + [Class A Principal] + [Class B Interest]
 Waterfall Coverage Check = [Collections Total] + [Recoveries] - [Total Waterfall Outflow]

Business Decisions Supported. Trustee report generation; credit enhancement adequacy monitoring; verifying no waterfall step is skipped or underfunded in a given period.

18.8 Stress Testing Dashboard

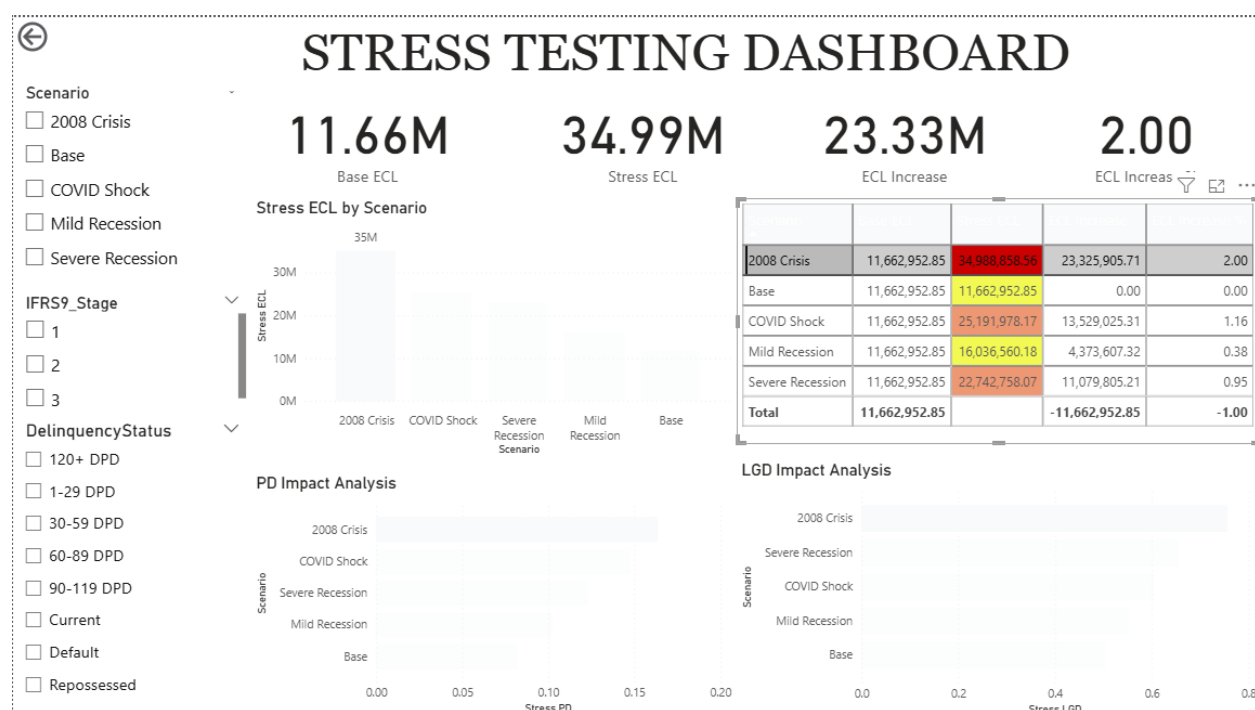


Figure 14: Stress Testing Dashboard

Fig. 17 — Stress Testing Dashboard.

Purpose. The direct Power BI consumer of stress_testing.py's output — giving the risk committee an interactive way to compare ECL under all five scenarios side by side.

KPIs. Base ECL (11.66M), Stress ECL (34.99M, shown for the currently selected/worst scenario), ECL Increase (23.33M), ECL Increase multiple (2.00x).

Visuals. Stress ECL by Scenario (bar chart ranking 2008 Crisis highest through Base lowest); a full scenario comparison table with conditional colour formatting (2008 Crisis flagged in red at 34,988,858.56 stress ECL against a 11,662,952.85 base — a 2.00x multiple; COVID Shock at 1.16x; Mild Recession at 0.38x; Severe Recession at 0.95x); PD Impact Analysis and LGD Impact Analysis horizontal bar charts by scenario.

Slicers. Scenario (multi-select), IFRS9_Stage, DelinquencyStatus.

Business Insights. The 2008 Crisis scenario doubling the portfolio's ECL (2.00x multiple, a 23.33M absolute increase) is the single number a board risk committee would fixate on — it directly translates into the additional capital buffer the institution would need to hold

if current conditions moved toward historical financial-crisis severity, and demonstrates the exact “what if” analysis IFRS 9’s forward-looking design was intended to enable.

DAX Measures (representative).

```
Stress ECL = SUM(stress_output[Stress_ECL])
ECL Increase Multiple = DIVIDE([Stress ECL], [Base ECL])
ECL Increase Absolute = [Stress ECL] - [Base ECL]
```

Business Decisions Supported. Internal Capital Adequacy Assessment Process (ICAAP) inputs; regulatory stress test submission; risk appetite and pricing policy calibration under adverse scenarios.

18.9 Investor Reporting Dashboard

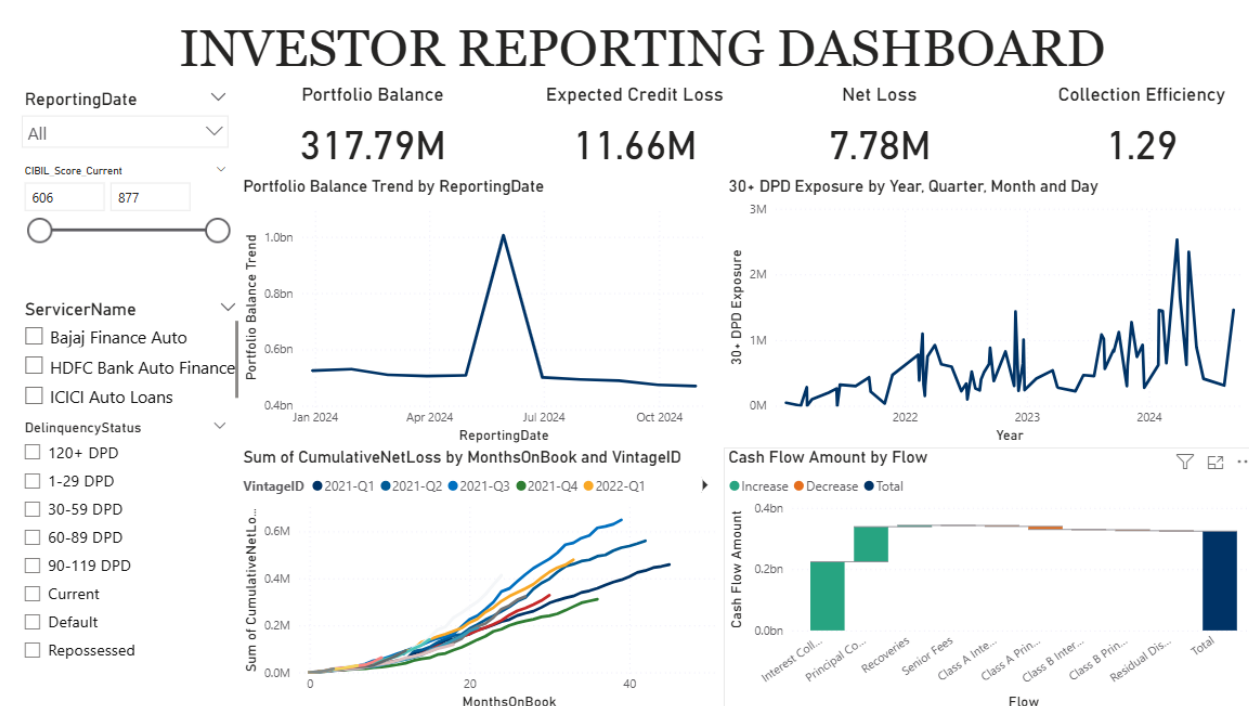


Figure 15: Investor Reporting Dashboard

Fig. 18 — Investor Reporting Dashboard.

Purpose. The externally-facing dashboard, structured to mirror what a trustee/servicer report delivers to ABS noteholders each month.

KPIs. Portfolio Balance (317.79M), Expected Credit Loss (11.66M), Net Loss (7.78M), Collection Efficiency (1.29).

Visuals. Portfolio Balance Trend by Reporting Date; 30+ DPD Exposure by Year/Quarter/Month/Day (drillable time hierarchy — allowing an investor or analyst to move from an annual view straight down to daily granularity without a separate report); Cumulative Net Loss by Months on Book and VintageID (the same vintage curve logic as the Vintage Dashboard, but framed here specifically for investor benchmarking against deal covenants); Cash Flow Amount by Flow (a compact version of the Waterfall Dashboard’s chart, giving investors the allocation picture without the full internal servicing detail).

Slicers. ReportingDate, CIBIL_Score_Current (range), ServicerName, DelinquencyStatus.

Business Insights. Presenting the CIBIL score range as an investor-facing slicer (606-877 in the current view) allows an investor to directly interrogate credit quality distribution themselves rather than relying solely on a summary statistic — a transparency feature that mirrors the kind of self-service investor reporting portals increasingly demanded by institutional ABS buyers.

DAX Measures (representative).

```
Investor Collection Efficiency = DIVIDE(SUM(fact_dynamic_loss[CollectionsTotal]), SUM(fact_dynamic_loss[Portfolio Balance]))  
30+ DPD Exposure (Investor View) = CALCULATE([Portfolio Balance], fact_auto_loans[DelinquencyDate] >= DATEADD(TODAY(), -30, DAY))
```

Business Decisions Supported. Monthly trustee/investor report distribution; covenant and trigger compliance monitoring; investor due diligence and secondary-market pricing support.

19. Business Insights

Table 11 — Business Insights Summary

1. **Stage 1 dominance masks concentrated risk.** 85.89% of portfolio balance sits in IFRS9 Stage 1, but Stage 3's disproportionate share of the 11.66M ECL confirms that provisioning risk is concentrated, not diffuse — a small population of impaired loans drives most of the reserve requirement.
2. **Early-stage delinquency has a high cure rate.** The roll-rate matrix shows loans in the 1-29 DPD bucket cure back to Current at a high rate, meaning early-intervention collections spend has a materially higher return than late-stage collections effort.
3. **90+ DPD is a near-certain loss event.** The 95% roll rate from 90-119 DPD into Default indicates this threshold is an effective, data-driven point to transition accounts from soft collections to recovery/legal workflows.
4. **Financial-crisis stress doubles ECL.** A 2.00x ECL multiple under the 2008 Crisis scenario quantifies the exact incremental capital buffer required if the institution's risk appetite framework targets resilience to historical crisis-level severity.
5. **PD is more macro-sensitive than LGD.** The consistently higher PD multipliers versus LGD multipliers across all five stress scenarios confirm that default frequency, not loss severity, is the primary driver of stressed ECL volatility — informing where risk mitigation (e.g. tighter underwriting vs. collateral policy) should be prioritised.
6. **Vintage curves reveal underwriting drift.** Plotting cumulative net loss rate by months-on-book across six origination cohorts allows immediate identification of any vintage underperforming its peers at the same age — the standard signal of a credit policy change requiring review.
7. **Pool factor confirms a maturing deal.** An origination-to-current-balance ratio of roughly 58% indicates the securitised pool is comfortably past its midpoint, relevant to clean-up call timing discussions.
8. **Net loss and CPR spikes moving together warrant investigation.** A correlated mid-year spike in both net loss and prepayment rate is a diagnostic flag for either a data quality issue or a broader liquidity/macro event, rather than an isolated credit deterioration.
9. **Credit enhancement is currently sufficient.** The waterfall reaching all the way to a positive residual distribution confirms realised losses have not yet impaired subordinated tranches, a reassuring signal for both Class B and equity holders.
10. **LTV distribution skews toward higher-risk collateral coverage.** The current-balance-by-LTV histogram's right skew flags a meaningful share of exposure at elevated loan-to-value ratios, directly relevant to expected loss severity if defaults occur.
11. **Geographic ECL concentration is directly queryable.** The Region and State summary views, sorted by calculated ECL descending, let a risk team immediately identify which geography is disproportionately driving provisioning need.
12. **Servicer performance is directly comparable.** The `vw_servicer_summary` view and Average Interest Rate by Servicer visual allow like-for-like benchmarking of PD, LGD, and ECL by servicing entity, informing servicer oversight and potential re-allocation decisions.
13. **Provisioning variance flags model/servicer drift.** The `Provision_Status` flag ("Under Provisioned" / "Over Provisioned") in `ifrs9_model.py` output directly identifies where the servicing system's held provision has drifted from the independently recalculated ECL — a key internal audit control point.

14. **Coverage ratio is a single-glance adequacy signal.** The portfolio-level Coverage Ratio (Calculated ECL / Portfolio Balance) gives a board-ready headline figure for provisioning adequacy without requiring loan-level detail.
15. **Top-20 highest-ECL loans form an actionable watchlist.** Both the IFRS 9 and Financial Crisis stress outputs generate a ranked Top-20 list, giving credit risk officers a direct, prioritised action list rather than an undifferentiated portfolio view.
16. **CIBIL score distribution supports underwriting policy review.** The Loan Count by CIBIL Score histogram on the Portfolio Overview dashboard makes credit quality distribution immediately visible, supporting cutoff and pricing tier discussions.
17. **Recovery concentration by vintage informs collections strategy.** The Cumulative Recoveries by VintageID pie chart shows recovery performance is not evenly distributed across cohorts, useful for calibrating recovery rate assumptions by origination period rather than applying a flat portfolio-wide rate.
18. **Excess spread trend is an early-warning indicator for trigger risk.** Because early-amortisation triggers in most ABS structures are tied to excess spread falling below a defined threshold, the Excess Spread trend line is effectively a leading indicator the trustee and structured finance team should monitor continuously.
19. **Collection efficiency variability by date signals operational, not just credit, risk.** The calendar heat-table view of Collection Efficiency exposes month-to-month operational performance variance that a purely credit-focused view would miss.
20. **Investor self-service via CIBIL slicer improves reporting transparency.** Allowing investors to filter the Investor Reporting Dashboard by CIBIL score range directly, rather than relying solely on a pre-aggregated summary statistic, reflects a more transparent, self-service reporting posture increasingly expected by institutional ABS buyers.

20. Challenges

20.1 Technical Challenges

Reconciling three independently computed sources of ECL — the loan-level ECL_Provision field carried in the source data, the Python-calculated Calculated_ECL, and the SQL stored-procedure-driven stress ECL — required careful attention to ensure all three used identical formula logic and rounding conventions; a mismatch of even a rounding convention (e.g. rounding at the loan level versus rounding only at the aggregate level) would produce visible, hard-to-diagnose discrepancies between the Python output and the Power BI dashboard totals.

20.2 Business/Domain Challenges

Calibrating the five stress scenario multipliers required grounding in real-world credit stress testing practice rather than arbitrary numbers — ensuring PD multipliers consistently exceeded LGD multipliers across every scenario (reflecting the established behavioural difference between default-frequency and loss-severity sensitivity to macro stress) was essential to keeping the model’s outputs directionally credible to a banking-literate reviewer.

20.3 Data Quality Challenges

Designing the validation layer to be strict enough to genuinely catch defects (out-of-range PD/LGD, invalid CIBIL scores, negative balances) while not being so rigid that it would reject legitimately messy real-world extracts required deliberate threshold-setting — the module raises a hard failure only for missing required columns, while all other checks are reported rather than blocking, mirroring how a real validation pipeline distinguishes between fatal schema errors and quality issues requiring analyst judgment.

20.4 Performance and Optimization Challenges

Ensuring the SQL views and indexes were designed around the columns actually used in Power BI’s filter and slicer context (IFRS9_Stage, DelinquencyStatus, ServicerName, date fields) rather than indexing indiscriminately required working backward from the dashboard design to the schema design — a deliberately iterative process rather than a purely top-down one.

21. Future Enhancements

- **Cloud Migration** — porting the MySQL schema to a managed cloud RDBMS (Azure SQL, AWS RDS) or a cloud data warehouse (Snowflake) to support larger loan books and concurrent multi-analyst access.
- **Databricks / Spark Layer** — re-implementing the Python validation and modelling layer on Databricks for distributed processing at multi-million-loan scale, beyond what a single-machine Pandas pipeline can efficiently handle.
- **Orchestration with Apache Airflow** — replacing the current manual three-script execution sequence with a DAG-based orchestration layer that schedules, monitors, and alerts on the monthly validation → ECL → stress testing → BI refresh pipeline.
- **Machine Learning PD Models** — replacing the currently sourced PD_Estimate field with a trained logistic regression or gradient-boosted model (e.g. XGBoost) built on the borrower, loan, and macro features already present in the schema, moving from a static estimate to a genuinely predictive model.
- **Real-Time / Near-Real-Time Streaming** — introducing a Kafka or equivalent event stream for delinquency status changes, enabling the DPD & Roll Rate dashboard to reflect intra-month movement rather than only monthly snapshots.
- **REST API Layer** — exposing the SQL views and stored procedures via a lightweight REST API (FastAPI/Flask) so the stress testing engine, in particular, could be called on-demand by other internal systems or a self-service risk committee tool.
- **CI/CD for the Data Pipeline** — adding automated testing (e.g. pytest against the validation module's rule logic) and a CI/CD pipeline to catch schema or logic regressions before they reach production data.
- **Generative AI-Assisted Narrative Reporting** — using an LLM to auto-draft the narrative commentary sections of the monthly investor report (e.g. explaining a net loss spike) from the underlying validated data, with an analyst reviewing and approving before distribution.
- **Predictive Analytics for Early Amortisation Trigger Risk** — extending the Dynamic Loss dashboard with a forward-looking projection of excess spread and cumulative loss trajectory against covenant thresholds, rather than only a backward-looking trend.

22. Conclusion

The Securitisation Risk Analytics Platform demonstrates, in a single coherent and fully reproducible build, the complete analytical chain that a bank or NBFC's structured finance and credit risk analytics function relies on every month: validated data feeding a regulator-grade IFRS 9 provisioning engine, that engine's output feeding a macroeconomic stress testing layer, and both flowing into a governed suite of Power BI dashboards purpose-built for nine distinct stakeholder audiences, from the CRO's desk to the ABS trustee's monthly report. Every design decision throughout the build — the choice of DECIMAL over floating-point types for monetary fields, the composite key on the vintage table reflecting its true grain, the deliberate calibration of PD multipliers above LGD multipliers in every stress scenario, the roll-rate matrix's direct translation into a collections resourcing decision — was made to mirror actual practice in institutional risk analytics rather than to produce a superficially impressive but practically hollow dashboard exercise. The platform is offered as direct, inspectable evidence of the ability to reason through, design, and build a production-representative banking analytics system end to end, using only freely available tooling, and is intended to stand on its own in a technical interview setting as a walkthrough-ready artefact.

23. References

- IFRS Foundation. *IFRS 9 Financial Instruments*. <https://www.ifrs.org/issued-standards/list-of-standards/ifrs-9-financial-instruments/>
- Basel Committee on Banking Supervision. *Basel Framework*. Bank for International Settlements. https://www.bis.org/basel_framework/
- Microsoft Learn. *Power BI Documentation*. <https://learn.microsoft.com/power-bi/>
- Microsoft Learn. *DAX Reference*. <https://learn.microsoft.com/dax/>
- MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>
- pandas development team. *pandas Documentation*. <https://pandas.pydata.org/docs/>
- NumPy Developers. *NumPy Documentation*. <https://numpy.org/doc/>
- Reserve Bank of India. *Master Direction - Reserve Bank of India (Securitisation of Standard Assets) Directions*. <https://www.rbi.org.in/>

24. Appendix

A.1 Folder Structure

```
Securitisation-Risk-Analytics/
├── Datasets/
│   ├── auto_loan_securitisation_data.csv
│   ├── dpd_snapshot_history.csv
│   ├── dynamic_loss_monthly.csv
│   └── static_pool_vintage_data.csv
├── SQL/
│   ├── 01_create_database.sql
│   ├── 02_create_tables.sql
│   ├── 03_import_data.sql
│   ├── 04_views.sql
│   └── 05_stored_procedures.sql
├── Python/
│   ├── data_validation.py
│   ├── ifrs9_model.py
│   ├── stress_testing.py
│   ├── Validation_Report.xlsx
│   ├── ECL_Output.xlsx
│   └── Stress_Test_Output.xlsx
├── PowerBI/
│   └── Securitisation_Risk_Analytics.pbix
├── Documentation/
│   └── Dashboard_Screenshots/
├── README.md
├── LICENSE
└── .gitignore
```

A.2 Technology Stack Summary

Table 12 — Technology Stack Summary

Layer	Technology	Role
Database	MySQL 8.0	System of record, views, stored procedures
Modelling	Python 3.12 (Pandas, NumPy, OpenPyXL)	Validation, IFRS 9 ECL, stress testing
Visualization	Microsoft Power BI	Dashboarding, DAX measures, Power Query
Version Control	Git / GitHub	Source control and portfolio hosting

A.3 Glossary

- **ABS** — Asset-Backed Security
- **CDR** — Conditional Default Rate (annualised)
- **CPR** — Conditional Prepayment Rate (annualised)
- **DPD** — Days Past Due
- **EAD** — Exposure at Default
- **ECL** — Expected Credit Loss

- **IFRS 9** — International Financial Reporting Standard 9 (Financial Instruments)
- **LGD** — Loss Given Default
- **LTV** — Loan-to-Value ratio
- **PD** — Probability of Default
- **SPV** — Special Purpose Vehicle
- **WAC** — Weighted Average Coupon

A.4 Abbreviations

CRO — Chief Risk Officer · NBFC — Non-Banking Financial Company · KPI — Key Performance Indicator · DAX — Data Analysis Expressions · ETL — Extract, Transform, Load · CI/CD — Continuous Integration / Continuous Deployment

A.5 SQL Scripts — Index

01_create_database.sql · 02_create_tables.sql · 03_import_data.sql · 04_views.sql · 05_stored_procedures.sql

A.6 Python Modules — Index

data_validation.py (→ Validation_Report.xlsx) · ifrs9_model.py (→ ECL_Output.xlsx) · stress_testing.py (→ Stress_Test_Output.xlsx)

A.7 Dashboard Index

Executive Summary · Portfolio Overview · IFRS-9 · DPD & Roll Rate · Vintage · Dynamic Loss · Waterfall · Stress Testing · Investor Reporting

A.8 GitHub Repository Structure

The project is structured for direct GitHub portfolio hosting with a top-level README.md (overview, architecture diagram, tech stack, setup instructions), separated SQL/, Python/, and PowerBI/ directories mirroring the three-layer architecture described in Section 13, and a Documentation/ folder holding dashboard screenshots and this report for recruiter and interviewer review.

End of Report — Securitisation Risk Analytics Platform, Garvit Mehta, July 2026